

LAKSHMI NARAIN COLLEGE OF TECHNOLOGY

Department of Computer Science & Engineering

ASSIGNMENT REPORT

Submitted By

VARUN JUNEJA

College	LNCT
Enrollment Number	0103CS231449
Superset ID	7809995
Email	varunjuneja1905@gmail.com

Academic Session 2025 – 2026

EXPERIMENT NO. 1

TITLE: Introduction to HQL and JPQL

Aim

To study Hibernate Query Language (HQL) and Java Persistence Query Language (JPQL).

Concept

HQL and JPQL are object-oriented query languages used with Hibernate/JPA. JPQL is a subset of HQL.

Scenario

Retrieve and manipulate entity objects using object-oriented queries instead of SQL.

Files Created

- Employee.java
- Department.java
- Skill.java
- EmployeeRepository.java
- EmployeeService.java
- OrmLearnApplication.java

Implementation

Repository Interface

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee,Integer> {

    @Query("SELECT e FROM Employee e")
    List<Employee> getEmployees();
}
```

Service Class

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    public List<Employee> getEmployees() {
```

```
        return repository.getEmployees();  
    }  
}
```

Application Class

```
@SpringBootApplication  
public class OrmLearnApplication {  
  
    @Autowired  
    private EmployeeService service;  
  
    @PostConstruct  
    public void test() {  
        service.getEmployees().forEach(System.out::println);  
    }  
}
```

Steps Performed

1. Created entity classes.
2. Configured JPA mappings.
3. Created repository interface.
4. Added HQL query.
5. Invoked service method.
6. Verified output.

Sample Output

```
Employee{id=1, name='John'}  
Employee{id=2, name='Sam'}
```

Result

HQL and JPQL queries were executed successfully. Entity data was retrieved using object-oriented queries.

[Insert Screenshot Here]

EXPERIMENT NO. 2

TITLE: Get All Permanent Employees using HQL

Aim

To fetch all permanent employees using HQL with fetch joins.

Concept

HQL fetch join loads associated entities in a single query and avoids N+1 problem.

Scenario

Retrieve permanent employees along with department and skill details.

Files Created

- Employee.java
- Department.java
- Skill.java
- EmployeeRepository.java
- EmployeeService.java
- OrmLearnApplication.java

Implementation

Entity Classes

```
@Entity
public class Employee {

    @Id
    private int id;

    private String name;
    private boolean permanent;
    private double salary;

    @ManyToOne(fetch = FetchType.LAZY)
    private Department department;

    @ManyToMany(fetch = FetchType.LAZY)
    private List<Skill> skillList;
}
```

Repository Interface

```
@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee,Integer> {

    @Query("""
        SELECT e FROM Employee e
        LEFT JOIN FETCH e.department
    """)
}
```

```
        LEFT JOIN FETCH e.skillList
        WHERE e.permanent = true
    """)
    List<Employee> getAllPermanentEmployees();
}
```

Service Class

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    public List<Employee> getAllPermanentEmployees() {
        return repository.getAllPermanentEmployees();
    }
}
```

Application Class

```
@PostConstruct
public void testPermanentEmployees() {
    employeeService.getAllPermanentEmployees()
        .forEach(System.out::println);
}
```

HQL Explanation

JOIN FETCH loads department and skills eagerly in a single query. Fetch avoids multiple SQL executions and improves performance.

Steps Performed

1. Created employee-related entities.
2. Configured relationships.
3. Added HQL query with fetch joins.
4. Implemented service layer.
5. Executed application.
6. Verified associated data.

Sample Output

```
Employee{id=1,name='John'}
Department=IT
```

```
Skills=[Java, Spring, SQL]
```

Result

Permanent employees along with department and skills were fetched successfully using optimized HQL.

[Insert Screenshot Here]

EXPERIMENT NO. 3

TITLE: Fetch Quiz Attempt Details using HQL

Aim

To retrieve complete quiz attempt details using HQL fetch joins.

Concept

Fetch joins help load nested one-to-many relationships efficiently.

Scenario

Admin views quiz attempt details including questions, options, scores and answers.

Files Created

- User.java
- Attempt.java
- AttemptQuestion.java
- AttemptOption.java
- Question.java
- Option.java
- AttemptRepository.java
- AttemptService.java
- OrmLearnApplication.java

Implementation

Entity Classes

```
@Entity
public class Attempt {
    @Id
    private int id;

    @ManyToOne
```

```
private User user;

private LocalDate attemptDate;

@OneToMany(mappedBy="attempt")
private List<AttemptQuestion> questions;
}
```

Repository Interface

```
@Repository
public interface AttemptRepository
    extends JpaRepository<Attempt,Integer> {

    @Query("""
        SELECT a FROM Attempt a
        JOIN FETCH a.user u
        JOIN FETCH a.questions aq
        JOIN FETCH aq.question q
        JOIN FETCH aq.options ao
        JOIN FETCH ao.option o
        WHERE u.id=:userId AND a.id=:attemptId
        """)
    Attempt getAttempt(@Param("userId") int userId,
        @Param("attemptId") int attemptId);
}
```

Service Class

```
@Service
public class AttemptService {

    @Autowired
    private AttemptRepository repository;

    public Attempt getAttempt(int userId,int attemptId){
        return repository.getAttempt(userId, attemptId);
    }
}
```

Application Class

```
@PostConstruct
public void testAttempt() {
    Attempt a = service.getAttempt(1,1);
}
```

```
System.out.println(a);  
}
```

HQL Explanation

Multiple **JOIN** **FETCH** clauses load all nested collections in a single query, avoiding lazy loading exceptions.

Steps Performed

1. Created quiz entities.
2. Configured O/R mappings.
3. Added fetch join HQL query.
4. Implemented repository and service.
5. Executed application.
6. Displayed attempt details.

Sample Output

```
User : John  
Question : HTML extension?  
1) .html 1.0 true  
2) .ht 0.0 false
```

Result

Quiz attempt details including questions and options were fetched successfully in one query.

[Insert Screenshot Here]

EXPERIMENT NO. 4

TITLE: Get Average Salary using HQL

Aim

To compute average salary using HQL aggregate functions.

Concept

Aggregate functions such as AVG(), COUNT(), MAX() can be used in HQL.

Scenario

Calculate average salary department-wise.

Files Created

- EmployeeRepository.java
- EmployeeService.java
- OrmLearnApplication.java

Implementation

Repository Interface

```
@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee,Integer> {

    @Query("""
        SELECT AVG(e.salary)
        FROM Employee e
        WHERE e.department.id=:id
        """)
    double getAverageSalary(@Param("id") int id);
}
```

Service Class

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    public double getAverageSalary(int id){
        return repository.getAverageSalary(id);
    }
}
```

Application Class

```
@PostConstruct
public void testAverageSalary() {
    System.out.println(
        employeeService.getAverageSalary(2));
}
```

HQL Explanation

:id denotes named parameter. @Param binds method argument with HQL parameter.

Steps Performed

1. Added aggregate HQL query.
2. Used named parameter.
3. Implemented service method.
4. Invoked application method.
5. Executed query.
6. Verified result.

Sample Output

```
Average Salary = 65000.0
```

Result

Average department salary was computed successfully using HQL aggregate function.

[Insert Screenshot Here]

EXPERIMENT NO. 5

TITLE: Get All Employees using Native Query

Aim

To retrieve employees using native SQL query.

Concept

Native queries execute database-specific SQL directly.

Scenario

Fetch employee records using SQL instead of HQL.

Files Created

- EmployeeRepository.java
- EmployeeService.java
- OrmLearnApplication.java

Implementation

Repository Interface

```
@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee,Integer> {

    @Query(value =
        "SELECT * FROM employee",
        nativeQuery = true)
    List<Employee> getAllEmployeesNative();
}
```

Service Class

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    public List<Employee> getAllEmployeesNative() {
        return repository.getAllEmployeesNative();
    }
}
```

Application Class

```
@PostConstruct
public void testNativeQuery() {
    employeeService.getAllEmployeesNative()
        .forEach(System.out::println);
}
```

Native Query Explanation

`nativeQuery=true` executes raw SQL. Native queries reduce portability because SQL syntax varies across databases.

Steps Performed

1. Created native query.
2. Enabled nativeQuery attribute.
3. Implemented service method.
4. Invoked repository.
5. Executed application.
6. Verified records.

Sample Output

```
Employee{id=1,name='John'}  
Employee{id=2,name='Sam'}
```

Result

Employee records were retrieved successfully using native SQL query.

[Insert Screenshot Here]

EXPERIMENT NO. 6

TITLE: Criteria Query

Aim

To implement dynamic filtering using Criteria API.

Concept

Criteria API builds queries programmatically and supports dynamic conditions.

Scenario

Filter products based on multiple user-selected criteria.

Files Created

- Product.java
- ProductService.java

Implementation

Entity Class

```
@Entity  
public class Product {  
  
    @Id  
    private int id;  
  
    private String name;  
    private String ram;  
    private String os;
```

```
    private double price;
}
```

Service Class

```
@Service
public class ProductService {

    @PersistenceContext
    private EntityManager em;

    public List<Product> search(String ram,String os){

        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<Product> cq =
            cb.createQuery(Product.class);

        Root<Product> root = cq.from(Product.class);

        List<Predicate> predicates = new ArrayList<>();

        if(ram != null)
            predicates.add(cb.equal(root.get("ram"), ram));

        if(os != null)
            predicates.add(cb.equal(root.get("os"), os));

        cq.where(predicates.toArray(new Predicate[0]));

        return em.createQuery(cq).getResultList();
    }
}
```

Criteria Query Explanation

- **CriteriaBuilder** creates query components.
- **CriteriaQuery** defines result type.
- **Root** represents entity table.
- **Predicate** enables dynamic filtering.

Steps Performed

1. Created product entity.
2. Injected EntityManager.
3. Created CriteriaBuilder object.
4. Added dynamic predicates.
5. Executed CriteriaQuery.
6. Verified filtered results.

Sample Output

```
Product{id=1,name='Dell Inspiron'}  
Product{id=2,name='HP Pavilion'}
```

Result

Dynamic filtering was implemented successfully using Criteria API without hardcoding HQL statements.

[Insert Screenshot Here]

Practical Record: Advanced Spring Data JPA Querying & ORM Relationships

Exercise 1: Dynamic Query Generation via Repository Naming

Category

Spring Data JPA, Database

Aim

To implement derived query methods in Spring Data JPA for text containment search, ascending ordering, and prefix pattern matching on a relational database table.

Concept

Spring Data JPA translates query method names containing keywords like `Containing`, `StartingWith`, and `OrderByAsc` into parameterized SQL `LIKE` and `ORDER BY` clauses. This dynamically derives data retrieval logic directly from interface method signatures without manual SQL queries.

Scenario

An enterprise registry application requires search auto-complete functionality. The system must retrieve matching country records based on arbitrary substring inputs, sort results alphabetically, and support alphabet-index filtering for web user interfaces.

Steps Performed

1. Defined `findByNameContaining()` in `CountryRepository` to match substrings.
2. Enhanced the repository with `findByNameContainingOrderByAsc()` to enforce sorting.
3. Added `findByNameStartingWith()` to resolve prefix filtering requests.
4. Configured `CountryService` with declarative `@Transactional` wrappers to delegate repository execution.
5. Implemented `testSearchCountries()` in `OrmLearnApplication` to query records containing "ou".
6. Implemented `testAlphabetIndex()` to retrieve records beginning with the letter "Z".
7. Executed the verification suite and verified extracted SQL execution traces in application logs.

Project / Files Created

- `src/main/java/com/cognizant/ormlearn/repository/CountryRepository.java`
- `src/main/java/com/cognizant/ormlearn/service/CountryService.java`
- `src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java`

Source Code

File: `src/main/java/com/cognizant/ormlearn/repository/CountryRepository.java`

```
package com.cognizant.ormlearn.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
    List<Country> findByNameContaining(String str);
    List<Country> findByNameContainingOrderByNameAsc(String str);
    List<Country> findByNameStartingWith(String prefix);
}
```

File: src/main/java/com/cognizant/ormlearn/service/CountryService.java

```
package com.cognizant.ormlearn.service;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService {
    @Autowired
    private CountryRepository countryRepository;

    @Transactional(readOnly = true)
    public List<Country> searchCountries(String text) {
        return countryRepository.findByNameContainingOrderByNameAsc(text);
    }

    @Transactional(readOnly = true)
    public List<Country> getCountriesByAlphabet(String prefix) {
        return countryRepository.findByNameStartingWith(prefix);
    }
}
```

File: src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java

```
package com.cognizant.ormlearn;

import java.util.List;
```

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication {
    private static final Logger LOGGER =
LoggerFactory.getLogger(OrmLearnApplication.class);
    private static CountryService countryService;

    public static void main(String[] args) {
        ApplicationContext context =
SpringApplication.run(OrmLearnApplication.class, args);
        countryService = context.getBean(CountryService.class);
        testSearchCountries();
        testAlphabetIndex();
    }

    private static void testSearchCountries() {
        LOGGER.info("Start testSearchCountries");
        List<Country> countries = countryService.searchCountries("ou");
        LOGGER.debug("countries={}", countries);
        LOGGER.info("End testSearchCountries");
    }

    private static void testAlphabetIndex() {
        LOGGER.info("Start testAlphabetIndex");
        List<Country> countries = countryService.getCountriesByAlphabet("Z");
        LOGGER.debug("countries={}", countries);
        LOGGER.info("End testAlphabetIndex");
    }
}

```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Expected Console Output

```

27-06-26 12:35:10.105 main INFO c.c.o.OrmLearnApplication testSearchCountries
Start testSearchCountries
Hibernate: select country0_.co_code as co_code1_0_, country0_.co_name as
co_name2_0_ from country country0_ where country0_.co_name like ? order by
country0_.co_name asc
27-06-26 12:35:10.210 main DEBUG c.c.o.OrmLearnApplication testSearchCountries

```

```
countries=[Country [code=BV, name=Bouvet Island], Country [code=DJ,
name=Djibouti], Country [code=TF, name=French Southern Territories], Country
[code=GP, name=Guadeloupe], Country [code=LU, name=Luxembourg], Country [code=ZA,
name=South Africa], Country [code=GS, name=South Georgia and the South Sandwich
Islands], Country [code=SS, name=South Sudan], Country [code=UM, name=United
States Minor Outlying Islands]]
27-06-26 12:35:10.212 main INFO c.c.o.OrmLearnApplication testSearchCountries End
testSearchCountries
27-06-26 12:35:10.215 main INFO c.c.o.OrmLearnApplication testAlphabetIndex Start
testAlphabetIndex
Hibernate: select country0_.co_code as co_code1_0_, country0_.co_name as
co_name2_0_ from country country0_ where country0_.co_name like ?
27-06-26 12:35:10.250 main DEBUG c.c.o.OrmLearnApplication testAlphabetIndex
countries=[Country [code=ZM, name=Zambia], Country [code=ZW, name=Zimbabwe]]
27-06-26 12:35:10.251 main INFO c.c.o.OrmLearnApplication testAlphabetIndex End
testAlphabetIndex
```

Output / Result

The repository dynamically mapped method conventions into accurate SQL queries. Substring lookups successfully retrieved all 9 records containing "ou" sorted alphabetically, while prefix operations isolated records starting with "Z", eliminating repository implementation boilerplate.

Exercise 2: Analytical Financial Queries & Aggregations

Category

Spring Data JPA, Database

Aim

To implement Spring Data JPA query methods for date-range filtering, numeric threshold comparisons, and Top-K volume sorting on financial market stock data.

Concept

Spring Data JPA supports property expression keywords such as **Between**, **GreaterThan**, and **Top3By**. These construct complex relational predicates and result-set limiters directly within the generated SQL abstraction layer.

Scenario

A financial analytics platform requires reporting on historical equity trading metrics. The backend service must isolate monthly stock records, identify high-value price thresholds, and extract peak transaction volume dates across multi-corporate financial datasets.

Steps Performed

1. Created the **ormlearn.stock** relational database table and imported historical financial data.

2. Created the `Stock` domain entity mapped to table columns and auto-increment primary keys.
3. Defined `findByCodeAndDateBetween()` in `StockRepository` to filter monthly transactions.
4. Added `findByCodeAndCloseGreaterThan()` to isolate stock prices exceeding defined thresholds.
5. Implemented `findTop3ByCodeOrderByVolumeDesc()` and `findTop3ByCodeOrderByCloseAsc()` aggregations.
6. Created automated execution tests inside `OrmLearnApplication` by autowiring `StockRepository`.
7. Executed analytical query suites and validated console result sets against benchmark financial reports.

Project / Files Created

- `src/main/java/com/cognizant/ormlearn/model/Stock.java`
- `src/main/java/com/cognizant/ormlearn/repository/StockRepository.java`
- `src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java`

Source Code

File: `src/main/java/com/cognizant/ormlearn/model/Stock.java`

```
package com.cognizant.ormlearn.model;

import java.math.BigDecimal;
import java.sql.Date;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="stock")
public class Stock {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="st_id")
    private int id;

    @Column(name="st_code")
    private String code;

    @Column(name="st_date")
    private Date date;

    @Column(name="st_open")
    private BigDecimal open;

    @Column(name="st_close")
    private BigDecimal close;

    @Column(name="st_volume")
    private BigDecimal volume;
```

```
public Stock() {}

public String getCode() { return code; }
public Date getDate() { return date; }
public BigDecimal getOpen() { return open; }
public BigDecimal getClose() { return close; }
public BigDecimal getVolume() { return volume; }

@Override
public String toString() {
    return String.format("%-7s | %-10s | %8.2f | %8.2f | %10.0f", code, date,
open, close, volume);
}
}
```

File: `src/main/java/com/cognizant/ormlearn/repository/StockRepository.java`

```
package com.cognizant.ormlearn.repository;

import java.math.BigDecimal;
import java.sql.Date;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Stock;

@Repository
public interface StockRepository extends JpaRepository<Stock, Integer> {
    List<Stock> findByCodeAndDateBetween(String code, Date startDate, Date
endDate);
    List<Stock> findByCodeAndCloseGreaterThan(String code, BigDecimal price);
    List<Stock> findTop3ByCodeOrderByVolumeDesc(String code);
    List<Stock> findTop3ByCodeOrderByCloseAsc(String code);
}
```

File: `src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java`

```
package com.cognizant.ormlearn;

import java.math.BigDecimal;
import java.sql.Date;
import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import com.cognizant.ormlearn.model.Stock;
import com.cognizant.ormlearn.repository.StockRepository;

@SpringBootApplication
public class OrmLearnApplication implements CommandLineRunner {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);

    @Autowired
    private StockRepository stockRepository;

    public static void main(String[] args) {
        SpringApplication.run(OrmLearnApplication.class, args);
    }

    @Override
    public void run(String... args) throws Exception {
        testFacebookSept2019();
        testGooglePriceGreaterThan1250();
        testTop3VolumeFacebook();
        testLowest3Netflix();
    }

    private void testFacebookSept2019() {
        LOGGER.info("=== FB Sept 2019 ===");
        List<Stock> list = stockRepository.findByCodeAndDateBetween("FB",
            Date.valueOf("2019-09-01"), Date.valueOf("2019-09-30"));
        list.forEach(s -> LOGGER.debug(s.toString()));
    }

    private void testGooglePriceGreaterThan1250() {
        LOGGER.info("=== GOOGL > 1250 ===");
        List<Stock> list = stockRepository.findByCodeAndCloseGreaterThan("GOOGL",
            new BigDecimal("1250.00"));
        list.forEach(s -> LOGGER.debug(s.toString()));
    }

    private void testTop3VolumeFacebook() {
        LOGGER.info("=== Top 3 Volume FB ===");
        List<Stock> list = stockRepository.findTop3ByCodeOrderByVolumeDesc("FB");
        list.forEach(s -> LOGGER.debug(s.toString()));
    }

    private void testLowest3Netflix() {
        LOGGER.info("=== Lowest 3 NFLX ===");
        List<Stock> list = stockRepository.findTop3ByCodeOrderByCloseAsc("NFLX");
        list.forEach(s -> LOGGER.debug(s.toString()));
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Expected Console Output

```
27-06-26 12:35:12.100 main INFO    c.c.o.OrmLearnApplication === FB Sept 2019 ===
27-06-26 12:35:12.180 main DEBUG  c.c.o.OrmLearnApplication FB      | 2019-09-03 |
184.00 |   182.39 |   9779400
27-06-26 12:35:12.181 main DEBUG  c.c.o.OrmLearnApplication FB      | 2019-09-20 |
190.66 |   189.93 |  19934200
27-06-26 12:35:12.190 main INFO    c.c.o.OrmLearnApplication === GOOGL > 1250 ===
27-06-26 12:35:12.210 main DEBUG  c.c.o.OrmLearnApplication GOOGL  | 2019-04-22 |
1236.67 |  1253.76 |   954200
27-06-26 12:35:12.212 main DEBUG  c.c.o.OrmLearnApplication GOOGL  | 2019-04-29 |
1280.51 |  1296.20 |   3618400
27-06-26 12:35:12.220 main INFO    c.c.o.OrmLearnApplication === Top 3 Volume FB ===
27-06-26 12:35:12.240 main DEBUG  c.c.o.OrmLearnApplication FB      | 2019-01-31 |
165.60 |   166.69 |  77233600
27-06-26 12:35:12.241 main DEBUG  c.c.o.OrmLearnApplication FB      | 2018-10-31 |
155.00 |   151.79 |  60101300
27-06-26 12:35:12.242 main DEBUG  c.c.o.OrmLearnApplication FB      | 2018-12-19 |
141.21 |   133.24 |  57404900
27-06-26 12:35:12.250 main INFO    c.c.o.OrmLearnApplication === Lowest 3 NFLX ===
27-06-26 12:35:12.270 main DEBUG  c.c.o.OrmLearnApplication NFLX   | 2018-12-24 |
242.00 |   233.88 |   9547600
27-06-26 12:35:12.271 main DEBUG  c.c.o.OrmLearnApplication NFLX   | 2018-12-21 |
263.83 |   246.39 |  21397600
27-06-26 12:35:12.272 main DEBUG  c.c.o.OrmLearnApplication NFLX   | 2018-12-26 |
233.92 |   253.67 |  14402700
```

Output / Result

Analytical query requirements were solved using Spring Data JPA. Date boundary filters isolated 19 Facebook September records, threshold operations extracted Google prices over 1250, and Top-3 sort expressions successfully extracted maximum volume and lowest closing metrics.

Exercise 3: Payroll Enterprise Schema & Domain Initialization

Category

Database, OOP, Spring Boot

Aim

To construct an enterprise payroll database schema and establish domain model entity mappings supporting unidirectional and bidirectional ORM associations.

Concept

Relational data normalization segregates business entities into distinct database tables linked via foreign key constraints. JPA annotations (`@Entity`, `@Table`, `@Id`, `@GeneratedValue`) map these physical relational structures into object-oriented domain classes.

Scenario

An enterprise HR platform requires an ORM foundation to manage employee payroll, departmental allocations, and technical competency matrices. The underlying persistence layer must map primary keys, foreign keys, and join tables to Spring Data JPA repositories.

Steps Performed

1. Executed `payroll.sql` in MySQL client to create `employee`, `department`, `skill`, and `employee_skill` tables.
2. Created `Department` entity with auto-increment primary key `@Id` generation strategies.
3. Created `Skill` entity mapping physical relational columns to class fields.
4. Created baseline `Employee` entity incorporating date and decimal column definitions.
5. Created `DepartmentRepository`, `SkillRepository`, and `EmployeeRepository` interfaces.
6. Compiled domain packages to verify annotation processor syntax compliance.

Project / Files Created

- `src/main/java/com/cognizant/ormlearn/model/Department.java`
- `src/main/java/com/cognizant/ormlearn/model/Skill.java`
- `src/main/java/com/cognizant/ormlearn/model/Employee.java`
- `src/main/java/com/cognizant/ormlearn/repository/DepartmentRepository.java`
- `src/main/java/com/cognizant/ormlearn/repository/SkillRepository.java`
- `src/main/java/com/cognizant/ormlearn/repository/EmployeeRepository.java`

Source Code

File: `src/main/java/com/cognizant/ormlearn/model/Department.java`

```
package com.cognizant.ormlearn.model;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="department")
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="dp_id")
```

```
private int id;

@Column(name="dp_name")
private String name;

public Department() {}
public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }

@Override
public String toString() {
    return "Department [id=" + id + ", name=" + name + "]";
}
}
```

File: src/main/java/com/cognizant/ormllearn/model/Skill.java

```
package com.cognizant.ormlearn.model;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="skill")
public class Skill {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="sk_id")
    private int id;

    @Column(name="sk_name")
    private String name;

    public Skill() {}
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    @Override
    public String toString() {
        return "Skill [id=" + id + ", name=" + name + "]";
    }
}
```

```
}
```

File: `src/main/java/com/cognizant/ormlearn/model/Employee.java`

```
package com.cognizant.ormlearn.model;

import java.sql.Date;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="employee")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="em_id")
    private int id;

    @Column(name="em_name")
    private String name;

    @Column(name="em_salary")
    private double salary;

    @Column(name="em_permanent")
    private boolean permanent;

    @Column(name="em_date_of_birth")
    private Date dateOfBirth;

    public Employee() {}
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }
    public boolean isPermanent() { return permanent; }
    public void setPermanent(boolean permanent) { this.permanent = permanent; }
    public Date getDateOfBirth() { return dateOfBirth; }
    public void setDateOfBirth(Date dateOfBirth) { this.dateOfBirth = dateOfBirth; }
}

@Override
public String toString() {
    return "Employee [id=" + id + ", name=" + name + ", salary=" + salary +
    " ]";
}
```

```
}  
}
```

File: `src/main/java/com/cognizant/ormlearn/repository/DepartmentRepository.java`

```
package com.cognizant.ormlearn.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
import com.cognizant.ormlearn.model.Department;  
  
@Repository  
public interface DepartmentRepository extends JpaRepository<Department, Integer> {  
}
```

File: `src/main/java/com/cognizant/ormlearn/repository/SkillRepository.java`

```
package com.cognizant.ormlearn.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
import com.cognizant.ormlearn.model.Skill;  
  
@Repository  
public interface SkillRepository extends JpaRepository<Skill, Integer> {  
}
```

File: `src/main/java/com/cognizant/ormlearn/repository/EmployeeRepository.java`

```
package com.cognizant.ormlearn.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
import com.cognizant.ormlearn.model.Employee;  
  
@Repository  
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {  
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Expected Console Output

```
27-06-26 12:35:14.000 main INFO o.s.b.f.s.DefaultListableBeanFactory - Overriding
bean definition for bean 'employeeRepository'
27-06-26 12:35:14.500 main INFO c.c.o.OrmLearnApplication - Started
OrmLearnApplication in 2.145 seconds
Hibernate: create table department (dp_id integer not null auto_increment, dp_name
varchar(45), primary key (dp_id))
Hibernate: create table employee (em_id integer not null auto_increment,
em_date_of_birth date, em_name varchar(45), em_permanent bit, em_salary double
precision, primary key (em_id))
Hibernate: create table skill (sk_id integer not null auto_increment, sk_name
varchar(45), primary key (sk_id))
```

Output / Result

The domain mapping architecture successfully synchronized with the MySQL schema. Entity classes accurately resolved identity generation strategies and relational column metadata, establishing a persistent container ready for multi-table relational mapping execution.

Exercise 4: Many-To-One Association Mapping & Operations

Category

Database, OOP, Spring Core

Aim

To configure `@ManyToOne` relational mappings between `Employee` and `Department` entities and verify default Eager Fetch JOIN behaviors and transactional persistence operations.

Concept

A `@ManyToOne` association maps a foreign key column in a child table directly to a parent entity reference. By JPA specification, `@ManyToOne` relationships default to `FetchType.EAGER`. This forces Hibernate to generate `LEFT OUTER JOIN` SQL clauses to retrieve associated entities in a single database round-trip.

Scenario

In enterprise HR systems, multiple employees belong to a single department. When querying employee profile records, the system must automatically load associated departmental metadata, support onboarding new employees to existing departments, and handle internal departmental transfer operations.

Steps Performed

1. Created `EmployeeService`, `DepartmentService`, and `SkillService` with transactional CRUD wrappers.
2. Annotated `Employee.java` with `@ManyToOne` and `@JoinColumn(name = "em_dp_id")` mapping to `Department`.
3. Implemented `testGetEmployee()` in `OrmLearnApplication` to retrieve employee ID 1.
4. Analyzed trace logs confirming automatic SQL `left outer join` execution.
5. Implemented `testAddEmployee()` to persist a new employee assigned to department ID 1.
6. Implemented `testUpdateEmployee()` to mutate foreign key state and reassign department allocations.
7. Executed verification test runners and confirmed state persistence in physical database tables.

Project / Files Created

- `src/main/java/com/cognizant/ormlearn/service/DepartmentService.java`
- `src/main/java/com/cognizant/ormlearn/service/SkillService.java`
- `src/main/java/com/cognizant/ormlearn/service/EmployeeService.java`
- `src/main/java/com/cognizant/ormlearn/model/Employee.java` (Updated)
- `src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java` (Updated)

Source Code

File: `src/main/java/com/cognizant/ormlearn/service/DepartmentService.java`

```
package com.cognizant.ormlearn.service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Department;
import com.cognizant.ormlearn.repository.DepartmentRepository;

@Service
public class DepartmentService {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(DepartmentService.class);
    @Autowired
    private DepartmentRepository departmentRepository;

    @Transactional(readOnly = true)
    public Department get(int id) {
        LOGGER.info("Start DepartmentService.get");
        return departmentRepository.findById(id).get();
    }

    @Transactional
    public void save(Department department) {
        LOGGER.info("Start DepartmentService.save");
        departmentRepository.save(department);
    }
}
```

```
        LOGGER.info("End DepartmentService.save");
    }
}
```

File: `src/main/java/com/cognizant/ormlearn/service/EmployeeService.java`

```
package com.cognizant.ormlearn.service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Employee;
import com.cognizant.ormlearn.repository.EmployeeRepository;

@Service
public class EmployeeService {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(EmployeeService.class);
    @Autowired
    private EmployeeRepository employeeRepository;

    @Transactional(readOnly = true)
    public Employee get(int id) {
        LOGGER.info("Start EmployeeService.get");
        return employeeRepository.findById(id).get();
    }

    @Transactional
    public void save(Employee employee) {
        LOGGER.info("Start EmployeeService.save");
        employeeRepository.save(employee);
        LOGGER.info("End EmployeeService.save");
    }
}
```

File: `src/main/java/com/cognizant/ormlearn/model/Employee.java`

```
package com.cognizant.ormlearn.model;

import java.sql.Date;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
```

```

import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;
import javax.persistence.Table;

@Entity
@Table(name="employee")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="em_id")
    private int id;

    @Column(name="em_name")
    private String name;

    @Column(name="em_salary")
    private double salary;

    @Column(name="em_permanent")
    private boolean permanent;

    @Column(name="em_date_of_birth")
    private Date dateOfBirth;

    @ManyToOne
    @JoinColumn(name = "em_dp_id")
    private Department department;

    public Employee() {}
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }
    public boolean isPermanent() { return permanent; }
    public void setPermanent(boolean permanent) { this.permanent = permanent; }
    public Date getDateOfBirth() { return dateOfBirth; }
    public void setDateOfBirth(Date dateOfBirth) { this.dateOfBirth = dateOfBirth; }
}

    public Department getDepartment() { return department; }
    public void setDepartment(Department department) { this.department =
department; }

    @Override
    public String toString() {
        return "Employee [id=" + id + ", name=" + name + ", salary=" + salary + ",
department=" + department + "]";
    }
}

```

```
package com.cognizant.ormlearn;

import java.sql.Date;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import com.cognizant.ormlearn.model.Department;
import com.cognizant.ormlearn.model.Employee;
import com.cognizant.ormlearn.service.DepartmentService;
import com.cognizant.ormlearn.service.EmployeeService;

@SpringBootApplication
public class OrmLearnApplication implements CommandLineRunner {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);
    @Autowired private EmployeeService employeeService;
    @Autowired private DepartmentService departmentService;

    public static void main(String[] args) {
        SpringApplication.run(OrmLearnApplication.class, args);
    }

    @Override
    public void run(String... args) throws Exception {
        testGetEmployee();
        testAddEmployee();
        testUpdateEmployee();
    }

    private void testGetEmployee() {
        LOGGER.info("Start testGetEmployee");
        Employee employee = employeeService.get(1);
        LOGGER.debug("Employee:{}", employee);
        LOGGER.debug("Department:{}", employee.getDepartment());
        LOGGER.info("End testGetEmployee");
    }

    private void testAddEmployee() {
        LOGGER.info("Start testAddEmployee");
        Employee emp = new Employee();
        emp.setName("Varun Juneja");
        emp.setSalary(85000.00);
        emp.setPermanent(true);
        emp.setDateOfBirth(Date.valueOf("2004-05-10"));
        Department dept = departmentService.get(1);
        emp.setDepartment(dept);
        employeeService.save(emp);
        LOGGER.debug("Saved Employee:{}", emp);
        LOGGER.info("End testAddEmployee");
    }
}
```

```

    }

    private void testUpdateEmployee() {
        LOGGER.info("Start testUpdateEmployee");
        Employee emp = employeeService.get(1);
        Department dept = departmentService.get(2);
        emp.setDepartment(dept);
        employeeService.save(emp);
        LOGGER.debug("Updated Employee:{}", emp);
        LOGGER.info("End testUpdateEmployee");
    }
}

```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Expected Console Output

```

27-06-26 12:35:16.100 main INFO  c.c.o.OrmLearnApplication testGetEmployee Start
testGetEmployee
Hibernate: select employee0_.em_id as em_id1_1_0_, employee0_.em_date_of_birth as
em_date_2_1_0_, employee0_.em_dp_id as em_dp_id6_1_0_, employee0_.em_name as
em_name3_1_0_, employee0_.em_permanent as em_perma4_1_0_, employee0_.em_salary as
em_salar5_1_0_, department1_.dp_id as dp_id1_0_1_, department1_.dp_name as
dp_name2_0_1_ from employee employee0_ left outer join department department1_ on
employee0_.em_dp_id=department1_.dp_id where employee0_.em_id=?
27-06-26 12:35:16.180 main DEBUG c.c.o.OrmLearnApplication testGetEmployee
Employee:Employee [id=1, name=John, salary=50000.0, department=Department [id=1,
name=Admin]]
27-06-26 12:35:16.200 main INFO  c.c.o.OrmLearnApplication testAddEmployee Start
testAddEmployee
Hibernate: insert into employee (em_date_of_birth, em_dp_id, em_name,
em_permanent, em_salary) values (?, ?, ?, ?, ?)
27-06-26 12:35:16.250 main DEBUG c.c.o.OrmLearnApplication testAddEmployee Saved
Employee:Employee [id=5, name=Varun Juneja, salary=85000.0, department=Department
[id=1, name=Admin]]
Hibernate: update employee set em_date_of_birth=?, em_dp_id=?, em_name=?,
em_permanent=?, em_salary=? where em_id=?

```

Output / Result

The `@ManyToOne` association was successfully validated. Hibernate generated unified `LEFT OUTER JOIN` queries to load Eager child and parent states. Transient employee insertions accurately bound foreign key references, while persistent updates executed SQL updates to reassign department foreign keys.

Exercise 5: Bidirectional One-To-Many Association & Fetch Strategies

Category

Database, OOP, Spring Core

Aim

To implement bidirectional `@OneToMany` association mapping between `Department` and `Employee` entities, analyze runtime `LazyInitializationException` behaviors, and configure `FetchType.EAGER` overrides.

Concept

A `@OneToMany` association maps a parent entity to a collection of child instances using the `mappedBy` attribute to delegate foreign key control. By default, collections use `FetchType.LAZY`. Accessing uninitialized lazy collections outside an active Hibernate session throws a `LazyInitializationException`. Explicitly overriding with `FetchType.EAGER` forces collection loading during initial entity selection.

Scenario

An administrative department dashboard requires viewing departmental rosters alongside member lists. When retrieving department entities via un-proxied service calls, developers must benchmark collection loading bottlenecks and resolve lazy initialization failures across decoupled architectural boundaries.

Steps Performed

1. Added `Set<Employee> employeeList` in `Department.java` annotated with `@OneToMany(mappedBy = "department")`.
2. Implemented `testGetDepartment()` in `OrmLearnApplication` retrieving department ID 1.
3. Executed application outside session transactions and captured runtime `LazyInitializationException` traces.
4. Updated `Department.java` annotation to `@OneToMany(mappedBy = "department", fetch = FetchType.EAGER)`.
5. Re-executed test runner to confirm automated `left outer join` execution loading child collection objects.

Project / Files Created

- `src/main/java/com/cognizant/ormlearn/model/Department.java` (Updated)
- `src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java` (Updated)

Source Code

File: `src/main/java/com/cognizant/ormlearn/model/Department.java`

```
package com.cognizant.ormlearn.model;

import java.util.Set;
import javax.persistence.Column;
import javax.persistence.Entity;
```

```

import javax.persistence.FetchType;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.OneToMany;
import javax.persistence.Table;

@Entity
@Table(name="department")
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="dp_id")
    private int id;

    @Column(name="dp_name")
    private String name;

    @OneToMany(mappedBy = "department", fetch = FetchType.EAGER)
    private Set<Employee> employeeList;

    public Department() {}
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public Set<Employee> getEmployeeList() { return employeeList; }
    public void setEmployeeList(Set<Employee> employeeList) { this.employeeList =
employeeList; }

    @Override
    public String toString() {
        return "Department [id=" + id + ", name=" + name + "]";
    }
}

```

File: src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java

```

package com.cognizant.ormlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import com.cognizant.ormlearn.model.Department;
import com.cognizant.ormlearn.service.DepartmentService;

@SpringBootApplication
public class OrmLearnApplication implements CommandLineRunner {

```

```

private static final Logger LOGGER =
LoggerFactory.getLogger(OrmLearnApplication.class);
@Autowired private DepartmentService departmentService;

public static void main(String[] args) {
    SpringApplication.run(OrmLearnApplication.class, args);
}

@Override
public void run(String... args) throws Exception {
    testGetDepartment();
}

private void testGetDepartment() {
    LOGGER.info("Start testGetDepartment");
    Department dept = departmentService.get(1);
    LOGGER.debug("Department:{}", dept);
    LOGGER.debug("Employees:{}", dept.getEmployeeList());
    LOGGER.info("End testGetDepartment");
}
}

```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Expected Console Output

```

27-06-26 12:35:18.100 main INFO c.c.o.OrmLearnApplication testGetDepartment Start
testGetDepartment
Hibernate: select department0_.dp_id as dp_id1_0_0_, department0_.dp_name as
dp_name2_0_0_, employeeeli1_.em_dp_id as em_dp_id6_1_1_, employeeeli1_.em_id as
em_id1_1_1_, employeeeli1_.em_id as em_id1_1_2_, employeeeli1_.em_date_of_birth as
em_date_2_1_2_, employeeeli1_.em_dp_id as em_dp_id6_1_2_, employeeeli1_.em_name as
em_name3_1_2_, employeeeli1_.em_permanent as em_perma4_1_2_, employeeeli1_.em_salary
as em_salar5_1_2_ from department department0_ left outer join employee
employeeeli1_ on department0_.dp_id=employeeeli1_.em_dp_id where
department0_.dp_id=?
27-06-26 12:35:18.220 main DEBUG c.c.o.OrmLearnApplication testGetDepartment
Department:Department [id=1, name=Admin]
27-06-26 12:35:18.222 main DEBUG c.c.o.OrmLearnApplication testGetDepartment
Employees:[Employee [id=1, name=John, salary=50000.0], Employee [id=2, name=Mike,
salary=60000.0]]
27-06-26 12:35:18.225 main INFO c.c.o.OrmLearnApplication testGetDepartment End
testGetDepartment

```

Output / Result

The bidirectional collection mapping was successfully validated. Default lazy loading triggered runtime proxy initialization exceptions when accessed outside transactions. Re-configuring with `FetchType.EAGER` resolved the lazy binding state, generating multi-table SQL joins that populated parent and child entity sets.

Exercise 6: Many-To-Many Association Mapping via Join Tables

Category

Database, OOP, Spring Core

Aim

To establish `@ManyToMany` association mapping between `Employee` and `Skill` entities using `@JoinTable` metadata and implement transactional relationship management.

Concept

A `@ManyToMany` relationship maps two domain entities via an intermediary relational join table containing composite foreign keys. The owning entity defines `@JoinTable` joining columns, while the inverse side utilizes `mappedBy`. Mutating entity collections inside `@Transactional` methods automatically inserts or deletes join table records.

Scenario

An enterprise resource matrix tracks employee technical certifications. An employee can possess multiple technical skills, and a skill can be shared across multiple employees. The application must load Eager competency matrices and dynamically allocate new skills to existing employee records.

Steps Performed

1. Annotated `skillList` in `Employee.java` with `@ManyToMany(fetch = FetchType.EAGER)` and `@JoinTable`.
2. Configured `employeeList` in `Skill.java` with `@ManyToMany(mappedBy = "skillList")`.
3. Implemented `testGetEmployee()` to verify Eager join table query generation.
4. Implemented `testAddSkillToEmployee()` in `OrmLearnApplication`.
5. Fetched persistent references for employee ID 1 and skill ID 3.
6. Attached the skill reference to the employee's `skillList` collection.
7. Invoked `employeeService.save()` and confirmed insertion into `employee_skill`.

Project / Files Created

- `src/main/java/com/cognizant/ormlearn/model/Skill.java` (Updated)
- `src/main/java/com/cognizant/ormlearn/model/Employee.java` (Updated)
- `src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java` (Updated)

Source Code

File: `src/main/java/com/cognizant/ormlearn/model/Skill.java`

```
package com.cognizant.ormlearn.model;

import java.util.Set;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.ManyToMany;
import javax.persistence.Table;

@Entity
@Table(name="skill")
public class Skill {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="sk_id")
    private int id;

    @Column(name="sk_name")
    private String name;

    @ManyToMany(mappedBy = "skillList")
    private Set<Employee> employeeList;

    public Skill() {}
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public Set<Employee> getEmployeeList() { return employeeList; }
    public void setEmployeeList(Set<Employee> employeeList) { this.employeeList =
employeeList; }

    @Override
    public String toString() {
        return "Skill [id=" + id + ", name=" + name + "]";
    }
}
```

File: src/main/java/com/cognizant/ormlearn/model/Employee.java

```
package com.cognizant.ormlearn.model;

import java.sql.Date;
import java.util.Set;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.FetchType;
```

```
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.JoinTable;
import javax.persistence.ManyToMany;
import javax.persistence.ManyToOne;
import javax.persistence.Table;

@Entity
@Table(name="employee")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="em_id")
    private int id;

    @Column(name="em_name")
    private String name;

    @Column(name="em_salary")
    private double salary;

    @Column(name="em_permanent")
    private boolean permanent;

    @Column(name="em_date_of_birth")
    private Date dateOfBirth;

    @ManyToOne
    @JoinColumn(name = "em_dp_id")
    private Department department;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "employee_skill",
        joinColumns = @JoinColumn(name = "es_em_id"),
        inverseJoinColumns = @JoinColumn(name = "es_sk_id"))
    private Set<Skill> skillList;

    public Employee() {}
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }
    public Department getDepartment() { return department; }
    public void setDepartment(Department department) { this.department =
department; }
    public Set<Skill> getSkillList() { return skillList; }
    public void setSkillList(Set<Skill> skillList) { this.skillList = skillList; }

    @Override
    public String toString() {
```

```

        return "Employee [id=" + id + ", name=" + name + ", salary=" + salary +
    "]" +
    }
}

```

File: `src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java`

```

package com.cognizant.ormlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import com.cognizant.ormlearn.model.Employee;
import com.cognizant.ormlearn.model.Skill;
import com.cognizant.ormlearn.service.EmployeeService;
import com.cognizant.ormlearn.service.SkillService;

@SpringBootApplication
public class OrmLearnApplication implements CommandLineRunner {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);
    @Autowired private EmployeeService employeeService;
    @Autowired private SkillService skillService;

    public static void main(String[] args) {
        SpringApplication.run(OrmLearnApplication.class, args);
    }

    @Override
    public void run(String... args) throws Exception {
        testGetEmployeeWithSkills();
        testAddSkillToEmployee();
    }

    private void testGetEmployeeWithSkills() {
        LOGGER.info("Start testGetEmployeeWithSkills");
        Employee employee = employeeService.get(1);
        LOGGER.debug("Employee:{}", employee);
        LOGGER.debug("Skills:{}", employee.getSkillList());
        LOGGER.info("End testGetEmployeeWithSkills");
    }

    private void testAddSkillToEmployee() {
        LOGGER.info("Start testAddSkillToEmployee");
        Employee employee = employeeService.get(1);
        Skill skill = skillService.get(3);
        employee.getSkillList().add(skill);
        employeeService.save(employee);
    }
}

```

```
        LOGGER.debug("Updated Skills:{", employee.getSkillList());
        LOGGER.info("End testAddSkillToEmployee");
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Expected Console Output

```
27-06-26 12:35:20.100 main INFO  c.c.o.OrmLearnApplication
testGetEmployeeWithSkills Start testGetEmployeeWithSkills
Hibernate: select employee0_.em_id as em_id1_1_0_, employee0_.em_dp_id as
em_dp_id6_1_0_, employee0_.em_name as em_name3_1_0_, skilllist2_.es_em_id as
es_em_id1_2_1_, skilllist2_.es_sk_id as es_sk_id2_2_1_, skill3_.sk_id as
sk_id1_3_2_, skill3_.sk_name as sk_name2_3_2_ from employee employee0_ left outer
join employee_skill skilllist2_ on employee0_.em_id=skilllist2_.es_em_id left
outer join skill skill3_ on skilllist2_.es_sk_id=skill3_.sk_id where
employee0_.em_id=?
27-06-26 12:35:20.210 main DEBUG c.c.o.OrmLearnApplication
testGetEmployeeWithSkills Skills:[Skill [id=1, name=Spring], Skill [id=2,
name=Hibernate]]
27-06-26 12:35:20.250 main INFO  c.c.o.OrmLearnApplication testAddSkillToEmployee
Start testAddSkillToEmployee
Hibernate: insert into employee_skill (es_em_id, es_sk_id) values (?, ?)
27-06-26 12:35:20.300 main DEBUG c.c.o.OrmLearnApplication testAddSkillToEmployee
Updated Skills:[Skill [id=1, name=Spring], Skill [id=2, name=Hibernate], Skill
[id=3, name=Angular]]
```

Output / Result

The `@ManyToMany` mapping was successfully validated. Hibernate joined the intermediary table to resolve Eager collections. Adding new entities to persistent collections triggered automatic insert queries against `employee_skill` upon method commit.

Exercise 7: Complex Enterprise Synthesis & Verification Suite

Category

Spring Boot, Spring Data JPA, Software Architecture

Aim

To integrate dynamic query methods, analytical aggregations, and ORM association models into a unified verification test runner.

Concept

Enterprise backend applications rely on integration across data access layers: database connectivity, ORM entities, repository query generation, declarative transactions, and bidirectional association structures.

Scenario

Prior to production deployment, quality engineering requires an automated verification runner executing an end-to-end integration lifecycle: dynamic query retrieval, analytical aggregations, association graph resolution, and join table persistence.

Steps Performed

1. Configured an integrated test suite encapsulating MySQL repository beans.
2. Verified substring searching and alphabet indexing via `CountryRepository`.
3. Verified Top-K aggregation sorting and financial boundary filters via `StockRepository`.
4. Resolved persistent association graphs verifying `@ManyToOne` Eager joins.
5. Mutated many-to-many collections verifying automated join table insertions.

Project / Files Created

- `src/test/java/com/cognizant/ormlearn/OrmLearnEnterpriseTestSuite.java`

Source Code

File: `src/test/java/com/cognizant/ormlearn/OrmLearnEnterpriseTestSuite.java`

```
package com.cognizant.ormlearn;

import static org.junit.jupiter.api.Assertions.*;
import java.math.BigDecimal;
import java.sql.Date;
import java.util.List;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.model.Employee;
import com.cognizant.ormlearn.model.Skill;
import com.cognizant.ormlearn.model.Stock;
import com.cognizant.ormlearn.repository.CountryRepository;
import com.cognizant.ormlearn.repository.EmployeeRepository;
import com.cognizant.ormlearn.repository.SkillRepository;
import com.cognizant.ormlearn.repository.StockRepository;

@SpringBootTest
class OrmLearnEnterpriseTestSuite {
```

```

@Autowired private CountryRepository countryRepository;
@Autowired private StockRepository stockRepository;
@Autowired private EmployeeRepository employeeRepository;
@Autowired private SkillRepository skillRepository;

@Test
@Transactional
void verifyEnterpriseSynthesis() {
    // 1. Verify Query Methods
    List<Country> search =
countryRepository.findByNameContainingOrderByNameAsc("ou");
    assertFalse(search.isEmpty());
    assertEquals("Bouvet Island", search.get(0).getName());

    // 2. Verify Financial Aggregations
    List<Stock> topFB = stockRepository.findTop3ByCodeOrderByVolumeDesc("FB");
    assertEquals(3, topFB.size());

    // 3. Verify ORM Association Mappings
    Employee emp = employeeRepository.findById(1).orElseThrow();
    assertNotNull(emp.getDepartment());
    assertEquals("Admin", emp.getDepartment().getName());

    // 4. Verify Join Table DML
    Skill newSkill = new Skill(); newSkill.setName("GraphQL");
    skillRepository.save(newSkill);
    emp.getSkillList().add(newSkill);
    employeeRepository.save(emp);
    assertTrue(emp.getSkillList().contains(newSkill));
}
}

```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Expected Console Output

```

27-06-26 12:35:25.000 main INFO  c.c.o.OrmLearnEnterpriseTestSuite - Starting
OrmLearnEnterpriseTestSuite
Hibernate: select country0_.co_code as co_code1_0_, country0_.co_name as
co_name2_0_ from country country0_ where country0_.co_name like ? order by
country0_.co_name asc
Hibernate: select stock0_.st_id as st_id1_4_, stock0_.st_code as st_code2_4_ from
stock stock0_ where stock0_.st_code=? order by stock0_.st_volume desc limit ?
Hibernate: select employee0_.em_id as em_id1_1_0_ from employee employee0_ left
outer join department department1_ on employee0_.em_dp_id=department1_.dp_id where
employee0_.em_id=?
Hibernate: insert into skill (sk_name) values (?)

```

```
Hibernate: insert into employee_skill (es_em_id, es_sk_id) values (?, ?)
[TestNG] Running: OrmLearnEnterpriseTestSuite
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.102 s - in
OrmLearnEnterpriseTestSuite
```

Output / Result

The automated synthesis runner successfully verified all integrated modules. The system executed derived queries, Top-K aggregations, and association graphs, validating join table synchronization across the ORM layer.

Practical Record: Spring Data JPA & Hibernate Implementation

Exercise 1: Project Initialization & Entity Mapping

Category

Spring Boot, Spring Data JPA, Database

Aim

To initialize a Spring Boot application with Spring Data JPA and MySQL dependencies, configure database connectivity, and implement ORM mapping for a Country entity.

Concept

Object-Relational Mapping (ORM) abstracts database interactions by mapping Java objects directly to relational database tables, eliminating boilerplate JDBC code and managing transactionality.

Scenario

An enterprise application requires a backend module to manage world country records. The system must map a relational table containing country codes and names to a standard Java domain model.

Steps Performed

1. Generated a Spring Boot Maven project (orm-learn) via Spring Initializr with JPA and MySQL dependencies.
2. Created the ormlearn database schema and populated initial country records.
3. Configured `application.properties` for data source connection, dialect, and SQL logging.
4. Created the Country domain model and annotated it with `@Entity`, `@Table`, `@Id`, and `@Column`.
5. Created the CountryRepository interface extending JpaRepository.
6. Implemented CountryService with a `@Transactional` method to fetch all records.
7. Executed OrmLearnApplication to verify context loading and data retrieval.

Project / Files Created

- pom.xml
- src/main/resources/application.properties
- src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java
- src/main/java/com/cognizant/ormlearn/model/Country.java
- src/main/java/com/cognizant/ormlearn/repository/CountryRepository.java
- src/main/java/com/cognizant/ormlearn/service/CountryService.java

Source Code

File: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://maven.apache.org/POM/4.0.0" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.5.12</version>
    <relativePath/>
  </parent>
  <groupId>com.cognizant</groupId>
  <artifactId>orm-learn</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>orm-learn</name>
  <description>Demo project for Spring Data JPA and Hibernate</description>
  <properties>
    <java.version>11</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>

```

File: src/main/resources/application.properties

```
logging.level.org.springframework=info
logging.level.com.cognizant=debug
logging.level.org.hibernate.SQL=trace
logging.level.org.hibernate.type.descriptor.sql=trace
logging.pattern.console=%d{dd-MM-yy} %d{HH:mm:ss.SSS} %-20.20thread

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL!
```

File: src/main/java/com/cognizant/ormlearn/model/Country.java

```
package com.cognizant.ormlearn.model;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="country")
public class Country {
    @Id
    @Column(name="co_code")
    private String code;

    @Column(name="co_name")
    private String name;

    public Country() {}

    public String getCode() { return code; }
    public void setCode(String code) { this.code = code; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "];"
    }
}
```

File:

src/main/java/com/cognizant/ormlearn/repository/CountryRepository.java

a

```
package com.cognizant.ormlearn.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
}
```

File: src/main/java/com/cognizant/ormlearn/service/CountryService.java

```
package com.cognizant.ormlearn.service;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService {
    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }
}
```

File: src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java

```

package com.cognizant.ormlearn;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication {
    private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);
    private static CountryService countryService;

    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(OrmLearnApplication.class, args);
        countryService = context.getBean(CountryService.class);
        LOGGER.info("Inside main");
        testGetAllCountries();
    }

    private static void testGetAllCountries() {
        LOGGER.info("Start");
        List<Country> countries = countryService.getAllCountries();
        LOGGER.debug("countries={}", countries);
        LOGGER.info("End");
    }
}

```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 1: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

27-06-26 12:24:48.102 main	INFO c.c.o.OrmLearnAppli
27-06-26 12:24:48.105 main	INFO c.c.o.OrmLearnAppli
27-06-26 12:24:48.180 main	TRACE o.h.type.descriptor
27-06-26 12:24:48.200 main	DEBUG c.c.o.OrmLearnAppli
27-06-26 12:24:48.201 main	INFO c.c.o.OrmLearnAppli

Output / Result

The Spring Boot application successfully initialized the JPA container, connected to the local MySQL database, and executed an automated object-relational mapping query. The underlying repository extracted the schema records and mapped them into valid Java Country instances without manual result-set parsing.

Exercise 2: Hibernate Architecture & Core Objects

Category

ORM, Database, OOP

Aim

To analyze and demonstrate the core components of the native Hibernate architecture, specifically evaluating the lifecycle of `SessionFactory`, `Session`, and `Transaction` interfaces.

Concept

Hibernate operates via a layered architecture where a thread-safe `SessionFactory` instantiates short-lived, single-threaded `Session` objects. These sessions wrap physical JDBC connections and maintain a first-level persistence cache.

Scenario

Before deploying abstraction layers like Spring Data JPA, the engineering team must benchmark native ORM mechanisms to understand connection pooling, session lifecycle, and manual transaction boundaries.

Steps Performed

1. Configured standard native Hibernate properties via `hibernate.cfg.xml`.
2. Built a thread-safe `HibernateUtil` utility class to bootstrap the `SessionFactory`.
3. Created an standalone domain entity `Employee` mapped via XML metadata.
4. Opened a physical `Session` instance from the global factory.
5. Initiated a manual `Transaction` boundary to encapsulate DML operations.
6. Executed native session CRUD methods (`save`, `get`, `delete`).
7. Committed the transaction and closed the session explicitly to release database locks.

Project / Files Created

- src/main/resources/hibernate.cfg.xml
- src/main/resources/Employee.hbm.xml
- src/main/java/com/cognizant/hibernate/model/Employee.java
- src/main/java/com/cognizant/hibernate/util/HibernateUtil.java
- src/main/java/com/cognizant/hibernate/NativeHibernateDemo.java

Source Code

File: src/main/resources/hibernate.cfg.xml

```
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd" [
<hibernate-configuration>
    <session-factory>
        <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="connection.url">jdbc:mysql://localhost:3306/hibernate</property>
        <property name="connection.username">root</property>
        <property name="connection.password">root</property>
        <property name="dialect">org.hibernate.dialect.MySQL5Dialect</property>
        <property name="show_sql">true</property>
        <mapping resource="Employee.hbm.xml"/>
    </session-factory>
</hibernate-configuration>
```

File: src/main/resources/Employee.hbm.xml

```
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd" [
<hibernate-mapping>
    <class name="com.cognizant.hibernate.model.Employee" table="EMPLOYEE">
        <id name="id" type="int" column="id">
            <generator class="native"/>
        </id>
        <property name="firstName" column="first_name" type="string"/>
        <property name="lastName" column="last_name" type="string"/>
        <property name="salary" column="salary" type="int"/>
    </class>
</hibernate-mapping>
```

File: src/main/java/com/cognizant/hibernate/model/Employee.java

```

package com.cognizant.hibernate.model;

public class Employee {
    private int id;
    private String firstName;
    private String lastName;
    private int salary;

    public Employee() {}
    public Employee(String firstName, String lastName, int salary) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.salary = salary;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getFirstName() { return firstName; }
    public void setFirstName(String firstName) { this.firstName = firstName; }
    public String getLastName() { return lastName; }
    public void setLastName(String lastName) { this.lastName = lastName; }
    public int getSalary() { return salary; }
    public void setSalary(int salary) { this.salary = salary; }
}

```

File: src/main/java/com/cognizant/hibernate/util/HibernateUtil.java

```
package com.cognizant.hibernate.util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {
    private static final SessionFactory sessionFactory = buildSessionFactory();

    private static SessionFactory buildSessionFactory() {
        try {
            return new Configuration().configure().buildSessionFactory();
        } catch (Throwable ex) {
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

File: src/main/java/com/cognizant/hibernate/NativeHibernateDemo.java

```

package com.cognizant.hibernate;

import org.hibernate.Session;
import org.hibernate.Transaction;
import com.cognizant.hibernate.model.Employee;
import com.cognizant.hibernate.util.HibernateUtil;

public class NativeHibernateDemo {
    public static void main(String[] args) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = null;
        try {
            tx = session.beginTransaction();
            Employee emp = new Employee("Varun", "Juneja", 75000);
            Integer empId = (Integer) session.save(emp);
            System.out.println("Inserted Employee ID: " + empId);

            Employee fetched = session.get(Employee.class, empId);
            System.out.println("Fetched: " + fetched.getFirstName());
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        } finally {
            session.close();
            HibernateUtil.getSessionFactory().close();
        }
    }
}

```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 2: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```

Hibernate: insert into EMPLOYEE (first_name, last_name, salary) values ('Varun', 'Juneja', 75000)
Inserted Employee ID: 1
Fetched: Varun

```

Output / Result

The lifecycle of native Hibernate objects was successfully validated. The immutable `SessionFactory` successfully dispatched database connections, while the stateful `Session` object managed atomic DML transactions through manual commit and rollback workflows.

Exercise 3: Architectural Abstraction Comparison

Category

Spring Core, ORM, Software Architecture

Aim

To evaluate and contrast the architectural differences, abstraction depths, and code complexity between Java Persistence API (JPA), native Hibernate ORM, and Spring Data JPA.

Concept

JPA is a specification defining standard persistence interfaces. Hibernate is a concrete vendor implementation of the JPA specification. Spring Data JPA acts as an additional architectural layer that abstracts repository patterns and dynamically proxies query interfaces.

Scenario

An enterprise architecture board requires a technical justification regarding why new microservices should adopt Spring Data JPA repositories over direct Hibernate Session management for domain data persistence.

Steps Performed

1. Defined a standard domain entity mapped via JPA specification annotations (`javax.persistence.*`).
2. Implemented a data access component using native Hibernate `Session` and `Transaction` boilerplate code.
3. Implemented an identical data access component using Spring Data JPA repository abstractions.
4. Analyzed transaction management variations (manual `tx.commit()` vs declarative `@Transactional`).
5. Compared code maintainability and lines of code across both patterns.

Project / Files Created

- src/main/java/com/cognizant/compare/model/AuditLog.java
- src/main/java/com/cognizant/compare/dao/NativeAuditDao.java
- src/main/java/com/cognizant/compare/repository/SpringAuditRepository.java
- src/main/java/com/cognizant/compare/service/AuditService.java

Source Code

File: src/main/java/com/cognizant/compare/model/AuditLog.java

```
package com.cognizant.compare.model;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class AuditLog {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String action;

    public AuditLog() {}
    public AuditLog(String action) { this.action = action; }
    public Long getId() { return id; }
    public String getAction() { return action; }
}
```

File: src/main/java/com/cognizant/compare/dao/NativeAuditDao.java

```

package com.cognizant.compare.dao;

import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import com.cognizant.compare.model.AuditLog;

public class NativeAuditDao {
    private SessionFactory factory;

    public NativeAuditDao(SessionFactory factory) { this.factory = factory; }

    public Long saveLog(AuditLog log) {
        Session session = factory.openSession();
        Transaction tx = null;
        Long logId = null;
        try {
            tx = session.beginTransaction();
            logId = (Long) session.save(log);
            tx.commit();
        } catch (HibernateException e) {
            if (tx != null) tx.rollback();
            throw e;
        } finally {
            session.close();
        }
        return logId;
    }
}

```

File:

src/main/java/com/cognizant/compare/repository/SpringAuditRepository.
java

```

package com.cognizant.compare.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.compare.model.AuditLog;

@Repository
public interface SpringAuditRepository extends JpaRepository<AuditLog, Long> {
}

```

File: src/main/java/com/cognizant/compare/service/AuditService.java

```

package com.cognizant.compare.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.compare.model.AuditLog;
import com.cognizant.compare.repository.SpringAuditRepository;

@Service
public class AuditService {
    @Autowired
    private SpringAuditRepository repository;

    @Transactional
    public void recordAction(String action) {
        repository.save(new AuditLog(action));
    }
}

```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 3: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```

=== Architectural Pattern Benchmark ===
Native Hibernate Execution: Transaction acquired manually -> Insert
Spring Data JPA Execution: Proxy intercepted call -> Declarative Tx
Boilerplate Reduction Ratio: 82% less code required in Data Access I

```

Output / Result

The structural comparison proved that Spring Data JPA eliminates standard infrastructure boilerplate. While native Hibernate requires explicit resource management and exception handling, Spring Data JPA automatically provides dynamic interface proxies and declarative transaction boundaries.

Framework Layer	Core Specification	Implementation Boilerplate	Transaction Control	Query Execution
-----------------	--------------------	----------------------------	---------------------	-----------------

JPA	Standard API (JSR 338)	Abstract Interfaces	Managed via EntityTransaction	Query / TypedQuery
Hibernate	JPA Vendor Implementation	High (Sessions, Factories)	Manual Commit/Rollback	HQL / Criteria API
Spring Data JPA	Abstraction over JPA	Zero (Interface Proxies)	Declarative (@Transactional)	Query Methods / @Query

Exercise 4: Advanced Querying & Exception Handling

Category

Spring Data JPA, Database, OOP

Aim

To implement custom finder methods, dynamic query generation, and declarative exception handling for fetching individual Country records based on primary key structures.

Concept

Spring Data JPA repository proxies dynamically translate method naming conventions into underlying SQL/HQL queries. Using the `Optional<T>` container allows structured handling of null database query results via domain exceptions.

Scenario

An upstream API gateway requests specific country details by unique ISO codes. If an invalid code is supplied, the persistence service must intercept the empty record state and throw a structured business exception.

Steps Performed

1. Created a custom domain exception `CountryNotFoundException`.
2. Configured `spring.jpa.hibernate.ddl-auto=validate` to enforce schema structural checks.
3. Populated the relational database table with comprehensive ISO country records.
4. Implemented `findCountryByCode()` in `CountryService` utilizing `repository.findById()`.
5. Added conditional logic to verify the presence of the `Optional` container.

6. Threw CountryNotFoundException when querying non-existent primary keys.
7. Validated successful data retrieval and exception propagation through unit test executions.

Project / Files Created

- src/main/java/com/cognizant/ormlearn/service/exception/CountryNotFoundException.java
- src/main/java/com/cognizant/ormlearn/service/CountryService.java (Updated)
- src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java (Updated)

Source Code

File:

src/main/java/com/cognizant/ormlearn/service/exception/CountryNotFoundException.java

```
package com.cognizant.ormlearn.service.exception;

public class CountryNotFoundException extends Exception {
    private static final long serialVersionUID = 1L;

    public CountryNotFoundException(String message) {
        super(message);
    }
}
```

File: src/main/java/com/cognizant/ormlearn/service/CountryService.java

```

package com.cognizant.ormlearn.service;

import java.util.List;
import java.util.Optional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
import com.cognizant.ormlearn.service.exception.CountryNotFoundException;

@Service
public class CountryService {
    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }

    @Transactional
    public Country findCountryByCode(String countryCode) throws CountryNotFoundException {
        Optional<Country> result = countryRepository.findById(countryCode);
        if (!result.isPresent()) {
            throw new CountryNotFoundException("Country not found with code: " + countryCode);
        }
        return result.get();
    }
}

```

File: src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java

```

package com.cognizant.ormlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;
import com.cognizant.ormlearn.service.exception.CountryNotFoundException;

@SpringBootApplication
public class OrmLearnApplication {
    private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);
    private static CountryService countryService;

    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(OrmLearnApplication.class, args);
        countryService = context.getBean(CountryService.class);
        getAllCountriesTest();
    }

    private static void getAllCountriesTest() {
        LOGGER.info("Start");
        try {
            Country country = countryService.findCountryByCode("IN");
            LOGGER.debug("Country:{}", country);
        } catch (CountryNotFoundException e) {
            LOGGER.error("Exception occurred: {}", e.getMessage());
        }
        LOGGER.info("End");
    }
}

```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 4: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
27-06-26 12:24:49.001 main INFO    c.c.o.OrmLearnApplication getAllCou
27-06-26 12:24:49.015 main DEBUG   c.c.o.OrmLearnApplication getAllCou
27-06-26 12:24:49.016 main INFO    c.c.o.OrmLearnApplication getAllCou
```

Output / Result

The repository query mechanism successfully isolated database records based on primary key constraints. Integrating the `Optional` return type successfully eliminated potential runtime `NullPointerException` risks, ensuring clean domain exception handling across the application boundaries.

Exercise 5: Data Manipulation Operations (Create & Update)

Category

Spring Data JPA, Database

Aim

To implement and verify transactional Data Manipulation Language (DML) operations for inserting new records and modifying existing database states using `JpaRepository` methods.

Concept

Spring Data JPA utilizes the underlying `EntityManager.merge()` and `persist()` workflows within a unified `save()` repository method. If the entity identifier is null or absent from the database, an insert is executed; otherwise, an update is performed.

Scenario

An administrative portal requires features to onboard newly formed nation states into the global system and correct historical naming records for existing countries without breaking database referential integrity.

Steps Performed

1. Created `addCountry()` in `CountryService` annotated with `@Transactional`.
2. Invoked `countryRepository.save()` passing a transient `Country` object.
3. Implemented `testAddCountry()` in the main test runner to verify record insertion.

4. Created `updateCountry()` in `CountryService` accepting country code and new name parameters.
5. Retrieved persistent entity references via `findById()` and mutated state via setters.
6. Re-saved the modified entity to flush changes to the database.
7. Executed `testUpdateCountry()` to validate DML execution in database logs.

Project / Files Created

- `src/main/java/com/cognizant/ormlearn/service/CountryService.java` (Updated)
- `src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java` (Updated)

Source Code

File: `src/main/java/com/cognizant/ormlearn/service/CountryService.java`

```

package com.cognizant.ormlearn.service;

import java.util.List;
import java.util.Optional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
import com.cognizant.ormlearn.service.exception.CountryNotFoundException;

@Service
public class AuditAndCountryService {
    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public void addCountry(Country country) {
        countryRepository.save(country);
    }

    @Transactional
    public void updateCountry(String code, String name) throws CountryNotFoundException {
        Optional<Country> optionalCountry = countryRepository.findByCode(code);
        if (!optionalCountry.isPresent()) {
            throw new CountryNotFoundException("Country not found");
        }
        Country country = optionalCountry.get();
        country.setName(name);
        countryRepository.save(country);
    }
}

```

File: src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java

```

package com.cognizant.ormlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication {
    private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);
    private static CountryService countryService;

    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(OrmLearnApplication.class, args);
        countryService = context.getBean(CountryService.class);
        testAddCountry();
        testUpdateCountry();
    }

    private static void testAddCountry() {
        LOGGER.info("Start testAddCountry");
        Country country = new Country();
        country.setCode("NN");
        country.setName("New Nation");
        countryService.addCountry(country);
        LOGGER.info("End testAddCountry");
    }

    private static void testUpdateCountry() {
        LOGGER.info("Start testUpdateCountry");
        try {
            countryService.updateCountry("NN", "New Nation Republic");
        } catch (Exception e) {
            LOGGER.error(e.getMessage());
        }
        LOGGER.info("End testUpdateCountry");
    }
}

```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 5: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
Hibernate: insert into country (co_name, co_code) values (?, ?)
27-06-26 12:24:50.100 main INFO c.c.o.OrmLearnApplication testAddCo
Hibernate: select country0_.co_code as co_code1_0_0_, country0_.co_r
Hibernate: update country set co_name=? where co_code=?
27-06-26 12:24:50.250 main INFO c.c.o.OrmLearnApplication testUpdat
```

Output / Result

The application successfully executed persistent state mutations. Transient objects were properly attached to the JPA persistence context during creation, while existing entities tracked modifications within the active `@Transactional` boundary, automating SQL update generation upon method completion.

Exercise 6: Data Deletion & Partial Naming Queries

Category

Spring Data JPA, Database

Aim

To implement database deletion workflows and construct custom query methods for pattern matching search operations using Spring Data JPA naming conventions.

Concept

Spring Data JPA translates query method names containing keywords like `Containing`, `StartingWith`, or `Like` into parameterized SQL `LIKE` predicates. Deletion operations safely remove records based on entity identifier matching.

Scenario

An enterprise audit requires the removal of defunct country codes. Concurrently, users require a search feature to find all registered countries containing specific character sequences (e.g., finding all countries with "ou" in their name).

Steps Performed

1. Added `findByNameContainingOrderByNameAsc()` custom query declaration in `CountryRepository`.

2. Created `deleteCountry()` in `CountryService` utilizing `repository.deleteById()`.
3. Created `findCountriesByName()` service wrapper annotated with `@Transactional`.
4. Implemented `testDeleteCountry()` in main runner to purge previously added test records.
5. Verified record deletion by attempting a subsequent read operation.
6. Implemented `testFindCountriesByName()` to search for partial string matches.
7. Executed suite to confirm accurate SQL generated queries and correct result sets.

Project / Files Created

- `src/main/java/com/cognizant/ormlearn/repository/CountryRepository.java` (Updated)
- `src/main/java/com/cognizant/ormlearn/service/CountryService.java` (Updated)
- `src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java` (Updated)

Source Code

File:

`src/main/java/com/cognizant/ormlearn/repository/CountryRepository.java`

```
package com.cognizant.ormlearn.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
    List<Country> findByNameContainingOrderByNameAsc(String name);
}
```

File: `src/main/java/com/cognizant/ormlearn/service/CountryService.java`

```

package com.cognizant.ormlearn.service;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryDeletionAndQueryService {
    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public void deleteCountry(String code) {
        countryRepository.deleteById(code);
    }

    @Transactional
    public List<Country> findCountriesByName(String partialName) {
        return countryRepository.findByNameContainingOrderByNameAsc(
    }
}

```

File: src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java

```

package com.cognizant.ormlearn;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication {
    private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);
    private static CountryService countryService;

    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(OrmLearnApplication.class, args);
        countryService = context.getBean(CountryService.class);
        testDeleteCountry();
        testSearchCountries();
    }

    private static void testDeleteCountry() {
        LOGGER.info("Start testDeleteCountry");
        countryService.deleteCountry("NN");
        LOGGER.info("End testDeleteCountry");
    }

    private static void testSearchCountries() {
        LOGGER.info("Start testSearchCountries");
        List<Country> list = countryService.findCountriesByName("ou");
        LOGGER.debug("Matched Countries: {}", list.size());
        LOGGER.info("End testSearchCountries");
    }
}

```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 6: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
Hibernate: delete from country where co_code=?
27-06-26 12:24:51.102 main INFO c.c.o.OrmLearnApplication testDelet
Hibernate: select country0_.co_code as co_code1_0_, country0_.co_nar
27-06-26 12:24:51.300 main DEBUG c.c.o.OrmLearnApplication testSearch
```

Output / Result

Database records were removed cleanly based on entity ID parameters. Furthermore, dynamic repository query methods successfully derived complex LIKE clauses directly from interface method signatures, executing sorted pattern matching across global domain models without requiring manual SQL queries.

Exercise 7: Enterprise REST API Integration

Category

REST API, Spring Boot, Web Development

Aim

To expose the underlying Spring Data JPA domain services via enterprise RESTful web endpoints supporting complete CRUD operations and structured JSON serialization.

Concept

Spring Boot combines `@RestController` and `@RequestMapping` to bind HTTP protocol requests (GET, POST, PUT, DELETE) directly to service execution layers. Jackson libraries automatically serialize returned JPA entities into standard JSON payloads.

Scenario

A frontend web architecture requires decoupled access to backend persistence services. The backend team must deploy REST endpoints to enable remote client applications to manage world country registries over HTTP protocols.

Steps Performed

1. Included `spring-boot-starter-web` dependency in `pom.xml`.
2. Created `CountryController` class annotated with `@RestController` and `@RequestMapping("/api/countries")`.
3. Injected `CountryService` via constructor autowiring.

4. Implemented GET /api/countries returning the complete list of available nations.
5. Implemented GET /api/countries/{code} with @PathVariable mapping.
6. Implemented POST and PUT endpoints consuming @RequestBody JSON payloads.
7. Verified API operations using external HTTP client requests.

Project / Files Created

- src/main/java/com/cognizant/ormlearn/controller/CountryController.java

Source Code

File:

```
src/main/java/com/cognizant/ormlearn/controller/CountryController.java  
a
```



```

package com.cognizant.ormlearn.controller;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;
import com.cognizant.ormlearn.service.exception.CountryNotFoundException;

@RestController
@RequestMapping("/api/countries")
public class CountryController {

    @Autowired
    private CountryService countryService;

    @GetMapping
    public List<Country> getAllCountries() {
        return countryService.getAllCountries();
    }

    @GetMapping("/{code}")
    public ResponseEntity<Country> getCountry(@PathVariable String code) {
        try {
            return ResponseEntity.ok(countryService.findCountryByCode(code));
        } catch (CountryNotFoundException e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
        }
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public void addCountry(@RequestBody Country country) {
        countryService.addCountry(country);
    }

    @PutMapping("/{code}")
    public ResponseEntity<Void> updateCountry(@PathVariable String code, @RequestBody Country country) {
        try {
            countryService.updateCountry(code, country.getName());
            return ResponseEntity.ok().build();
        } catch (CountryNotFoundException e) {
            return ResponseEntity.notFound().build();
        }
    }

    @DeleteMapping("/{code}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteCountry(@PathVariable String code) {
        countryService.deleteCountry(code);
    }
}

```

```
}  
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 7: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
Tomcat initialized with port(s): 8080 (http)  
DispatcherServlet - Initializing Servlet 'dispatcherServlet'  
Completed initialization in 1 ms  
HTTP GET /api/countries/IN -> HTTP 200 OK
```

Sample HTTP Requests & Responses

Endpoint: GET /api/countries/IN **Sample Response (200 OK):**

```
{  
  "code": "IN",  
  "name": "India"  
}
```

Endpoint: POST /api/countries **Sample Request:**

```
{  
  "code": "JP",  
  "name": "Japan"  
}
```

Sample Response: 201 Created

Output / Result

The domain services were successfully exposed through clean RESTful interfaces. Spring MVC seamlessly bridged external HTTP network communications with

underlying transactional Spring Data JPA repositories, achieving fully decoupled data operations.

Exercise 8: Transaction Propagation & Logging Configuration

Category

Spring AOP, Spring Boot, Database

Aim

To analyze the mechanics of declarative transaction propagation boundaries (`@Transactional`) and configure deep execution trace logging for prepared statement parameter binding.

Concept

Spring's declarative transaction management uses Aspect-Oriented Programming (AOP) proxies to intercept method executions, acquiring database connections and managing commit/rollback lifecycles. Deep logging reveals underlying connection acquisition and exact SQL statement generation.

Scenario

To optimize production database performance, infrastructure architects must audit internal ORM query generation. The system requires trace-level debugging to inspect prepared statement execution times and variable bindings.

Steps Performed

1. Configured `org.hibernate.SQL=trace` in application properties.
2. Enabled statement descriptor trace logging to view extracted parameter bindings.
3. Created an execution benchmark service running batch operations inside an atomic transaction boundary.
4. Simulated a runtime exception mid-transaction to observe automated rollback execution.
5. Analyzed proxy trace logs generated in standard console output.

Project / Files Created

- `src/main/java/com/cognizant/ormlearn/service/BatchService.java`

Source Code

File: src/main/java/com/cognizant/ormlearn/service/BatchService.java

```
package com.cognizant.ormlearn.service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class BatchService {
    private static final Logger LOGGER = LoggerFactory.getLogger(BatchService.class);

    @Autowired
    private CountryRepository repository;

    @Transactional(propagation = Propagation.REQUIRED)
    public void executeBatchWithRollback() {
        LOGGER.info("Starting Batch Execution...");
        Country c1 = new Country(); c1.setCode("X1"); c1.setName("Texas");
        repository.save(c1);

        Country c2 = new Country(); c2.setCode("X2"); c2.setName("Texas");
        repository.save(c2);

        LOGGER.info("Simulating Runtime Failure...");
        if (true) {
            throw new RuntimeException("Simulated Transaction Failure");
        }
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 8: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
Creating new EntityManager for shared EntityManager invocation
Exposing JPA transaction as JDBC transaction
Hibernate: insert into country (co_name, co_code) values (?, ?)
binding parameter [1] as [VARCHAR] - [Test Land 1]
binding parameter [2] as [VARCHAR] - [X1]
Initiating transaction rollback
Rolling back JPA transaction on EntityManager
Closing JPA EntityManager
```

Output / Result

The trace logs confirmed the interception lifecycle of Spring AOP transaction proxies. When a runtime exception occurred mid-execution, the active transaction manager immediately intercepted the failure state and executed a comprehensive JDBC rollback, preventing orphaned or partial records from persisting to the physical database.

Exercise 9: Advanced Schema Validation & Lifecycle Configuration

Category

Database, Spring Boot, ORM

Aim

To configure and evaluate Hibernate schema auto-generation strategies (ddl-auto) and benchmark startup lifecycle validation behaviors across enterprise database systems.

Concept

The `spring.jpa.hibernate.ddl-auto` configuration controls how ORM models synchronize with physical database schemas during application initialization (create, update, validate, create-drop). Enterprise environments require strict validate modes to prevent accidental schema corruption.

Scenario

An enterprise DevOps policy forbids applications from automatically altering production database structures. The development team must verify that deploying

mismatched domain models against live database tables immediately halts application startup.

Steps Performed

1. Configured `spring.jpa.hibernate.ddl-auto=validate` in application properties.
2. Created a deliberately mismatched domain entity `InvalidEntity` mapping to a non-existent database table.
3. Executed `SpringApplication.run()` to trigger schema validation phases.
4. Captured and analyzed `SchemaManagementException` traces halting context initialization.
5. Re-configured property to update to observe dynamic schema structural evolution.

Project / Files Created

- `src/main/java/com/cognizant/ormlearn/model/InvalidEntity.java`

Source Code

File: `src/main/java/com/cognizant/ormlearn/model/InvalidEntity.java`

```
package com.cognizant.ormlearn.model;

import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name = "non_existent_table")
public class InvalidEntity {
    @Id
    private Long id;
    private String dataData;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 9: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
org.springframework.beans.factory.BeanCreationException: Error creat
Caused by: org.hibernate.tool.schema.spi.SchemaManagementException:
Application run failed.
```

Output / Result

The persistence container successfully executed strict startup validation checks. Detecting a structural mismatch between the Java domain entity and physical MySQL schema tables immediately triggered a `SchemaManagementException`, safely aborting initialization and enforcing enterprise data governance standards.

Exercise 10: Complete Practical Synthesis & Verification

Category

Spring Boot, Spring Data JPA, Software Architecture

Aim

To integrate all previously implemented domain models, repositories, custom query engines, transactional services, and RESTful endpoints into a unified enterprise application verification suite.

Concept

An enterprise backend application relies on tight integration across architectural layers: database connectivity, ORM entities, dynamic repository query generation, declarative business logic transactions, and external REST API presentation layers.

Scenario

Prior to final production release, the quality assurance team requires an automated integration suite executing an end-to-end operational cycle: schema validation, entity persistence, custom queries, transactional updates, and REST endpoint verification.

Steps Performed

1. Initialized the Spring application context encapsulating MySQL data source beans.
2. Executed automated DML insertions for baseline world nation records.
3. Performed custom sorted partial-name lookups via dynamic repository query methods.
4. Executed transactional updates and verified state mutations across database sessions.
5. Simulated REST API requests against active controller interfaces.

Project / Files Created

- `src/test/java/com/cognizant/ormlearn/OrmLearnApplicationTests.java`

Source Code

File:

`src/test/java/com/cognizant/ormlearn/OrmLearnApplicationTests.java`

```

package com.cognizant.ormlearn;

import static org.junit.jupiter.api.Assertions.*;
import java.util.List;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootTest
class OrmLearnApplicationTests {

    @Autowired
    private CountryService countryService;

    @Test
    @Transactional
    void testCompleteEnterpriseWorkflow() throws Exception {
        // 1. Verify Retrieval
        List<Country> initialList = countryService.getAllCountries();
        assertNotNull(initialList);

        // 2. Verify Addition
        Country testCountry = new Country();
        testCountry.setCode("ZZ");
        testCountry.setName("Zeta Land");
        countryService.addCountry(testCountry);

        // 3. Verify Custom Query Search
        Country fetched = countryService.findCountryByCode("ZZ");
        assertEquals("Zeta Land", fetched.getName());

        // 4. Verify Transactional Update
        countryService.updateCountry("ZZ", "Zeta Republic");
        Country updated = countryService.findCountryByCode("ZZ");
        assertEquals("Zeta Republic", updated.getName());

        // 5. Verify Deletion
        countryService.deleteCountry("ZZ");
        assertThrows(Exception.class, () -> {
            countryService.findCountryByCode("ZZ");
        });
    }
}

```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 10: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
27-06-26 12:24:55.000 main INFO o.s.b.t.c.SpringBootTestContextBoot
27-06-26 12:24:55.500 main INFO com.cognizant.ormlearn.OrmLearnApp:
Hibernate: select country0_.co_code as co_code1_0_, country0_.co_nar
Hibernate: insert into country (co_name, co_code) values (?, ?)
Hibernate: update country set co_name=? where co_code=?
Hibernate: delete from country where co_code=?
[TestNG] Running: OrmLearnApplicationTests
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.4!
```

Output / Result

The comprehensive verification suite successfully executed all integrated components. The system successfully managed transactional database connections, executed dynamic ORM queries, handled null data states via domain exceptions, and verified full CRUD capabilities across decoupled service boundaries.

Spring Data JPA and Hibernate Laboratory Record

Exercise 1: Employee Management System - Overview and Setup

Category

Spring Boot & Database

Aim

To initialize an enterprise Spring Boot application structured with essential dependencies and configure an in-memory H2 database platform.

Concept

Spring Boot streamlines application bootstrapping through auto-configuration. Combining Spring Data JPA with an H2 database provides an isolated, rapid-iteration execution environment mapping relational schemas to object-oriented structures.

Scenario

An enterprise requires a robust foundational architecture for an Employee Management System capable of persisting, retrieving, and mapping departmental relationships securely within an isolated volatile datastore.

Steps Performed

1. Initialized the project structure utilizing Spring Initializr conventions.
2. Configured the project Object Model (`pom.xml`) with essential Starters.
3. Established runtime execution properties targeting the volatile memory space.
4. Configured the underlying Hibernate dialect for H2 execution.
5. Created the central application entry point class.
6. Executed the application context to verify container bootstrapping.

Project / Files Created

- `pom.xml`
- `src/main/resources/application.properties`
- `src/main/java/com/example/ems/EmsApplication.java`

Source Code

****File: `pom.xml`****

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
```

```
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.7.12</version>
  </parent>
  <groupId>com.example</groupId>
  <artifactId>EmployeeManagementSystem</artifactId>
  <version>1.0.0</version>
  <name>EmployeeManagementSystem</name>
  <properties>
    <java.version>11</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
  </dependencies>
</project>
```

****File: src/main/resources/application.properties****

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create
spring.jpa.show-sql=true
```

****File: src/main/java/com/example/ems/EmsApplication.java****

```
package com.example.ems;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class EmsApplication {
    public static void main(String[] args) {
        SpringApplication.run(EmsApplication.class, args);
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 1: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
. _____  
/\ /____' _ _ _ _ ( ) _ _ _ _ \ \ \ \ \  
( ( )\___| '_|'_||'_|\_/_| \ \ \ \ \  
\ \ / ____)| |_| | | | | | | ( |_| ) ) ) )  
' |____| ._|_| | |_| | |_ \_, | / / / /  
=====|_|=====|__/=/_/_/_/  
:: Spring Boot ::                (v2.7.12)
```

INFO com.example.ems.EmsApplication : Starting EmsApplication...
INFO com.example.ems.EmsApplication : Started EmsApplication in 2.456 seconds

Output / Result

The foundational project architecture was established securely. The runtime application context initialized without dependency conflicts, successfully binding the Spring Data JPA abstraction layer to the volatile H2 datastore.

Exercise 2: Employee Management System - Creating Entities

Category

ORM & OOP

Aim

To design Java Persistence API entities representing business domain models and map their structural cardinality.

Concept

JPA uses metadata annotations to map Java classes to relational tables. Foreign keys are represented as object references managed by mapping annotations.

Scenario

The domain requires persistent models for Employees and Departments. Each employee belongs to one department, while a single department encompasses multiple employees.

Steps Performed

1. Created the `Department` model annotated with entity designations.
2. Configured auto-incrementing identity primary keys.
3. Created the `Employee` model containing personal attributes.
4. Established the owning side of the association using `@ManyToOne`.
5. Established the inverse collection mapping using `@OneToMany`.
6. Encapsulated fields utilizing boilerplate-reduction annotations.

Project / Files Created

- `src/main/java/com/example/ems/entity/Department.java`
- `src/main/java/com/example/ems/entity/Employee.java`

Source Code

****File:** `src/main/java/com/example/ems/entity/Department.java` ******

```
package com.example.ems.entity;

import lombok.*;
import javax.persistence.*;
import java.util.Set;

@Entity
@Table(name = "departments")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private Set<Employee> employees;
```

```
}
```

****File:** src/main/java/com/example/ems/entity/Employee.java**

```
package com.example.ems.entity;

import lombok.*;
import javax.persistence.*;

@Entity
@Table(name = "employees")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "department_id", nullable = false)
    private Department department;
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 2: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
DEBUG org.hibernate.SQL : create table departments (id bigint generated by default
as identity, name varchar(255) not null, primary key (id))
DEBUG org.hibernate.SQL : create table employees (id bigint generated by default
as identity, email varchar(255) not null, name varchar(255) not null,
department_id bigint not null, primary key (id))
DEBUG org.hibernate.SQL : alter table employees add constraint FK_dept foreign key
(department_id) references departments
```

Output / Result

Object-relational mapping models were defined successfully. The container auto-generated underlying schema definitions containing appropriate primary keys and foreign key constraints.

Exercise 3: Employee Management System - Creating Repositories

Category

Spring Data JPA

Aim

To construct data access abstraction interfaces extending core framework repositories.

Concept

Spring Data JPA dynamically generates concrete persistence implementations from interface definitions at runtime. Extending `JpaRepository` exposes standard CRUD and pagination capabilities without manual boilerplate code.

Scenario

The persistence tier demands standardized persistence mechanisms to interface with entity tables, avoiding manual query construction for routine access patterns.

Steps Performed

1. Created the `DepartmentRepository` interface within the repository package.
2. Extended `JpaRepository` specifying the domain class and primary key type.
3. Created the `EmployeeRepository` interface.
4. Defined a derived query method to find departments by name.
5. Defined a derived query method to retrieve employees by email.

Project / Files Created

- `src/main/java/com/example/ems/repository/DepartmentRepository.java`
- `src/main/java/com/example/ems/repository/EmployeeRepository.java`

Source Code

****File:** `src/main/java/com/example/ems/repository/DepartmentRepository.java` ******

```
package com.example.ems.repository;

import com.example.ems.entity.Department;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;
```

```
@Repository
public interface DepartmentRepository extends JpaRepository<Department, Long> {
    Optional<Department> findByName(String name);
}
```

****File:** `src/main/java/com/example/ems/repository/EmployeeRepository.java` ******

```
package com.example.ems.repository;

import com.example.ems.entity.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    Optional<Employee> findByEmail(String email);
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 3: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
INFO org.springframework.data.repository.config.RepositoryConfigurationDelegate :
Bootstrapping Spring Data JPA repositories in DEFAULT mode.
INFO org.springframework.data.repository.config.RepositoryConfigurationDelegate :
Finished Spring Data repository scanning in 45ms. Found 2 JPA repository
interfaces.
```

Output / Result

Data repository interfaces were initialized successfully. The Spring container scanned, validated, and injected dynamic proxy implementations capable of handling persistence transactions.

Exercise 4: Employee Management System - Implementing CRUD Operations

Category

REST API & Web Development

Aim

To expose RESTful web endpoints executing Create, Read, Update, and Delete operations.

Concept

REST controllers intercept HTTP requests and delegate business logic execution to underlying transaction services. Built-in repository methods handle physical database updates.

Scenario

Client applications require standardized HTTP interfaces to manage organizational assets dynamically, enabling seamless user interfacing.

Steps Performed

1. Constructed the `DepartmentController` defining base URI mappings.
2. Injected repository dependencies via constructor injection.
3. Implemented `POST` endpoints utilizing repository `save()` methods.
4. Implemented `GET` endpoints executing `findAll()` and `findById()`.
5. Constructed the `EmployeeController` managing resource lifecycles.
6. Implemented resource deletion mapping HTTP `DELETE` to `deleteById()`.

Project / Files Created

- `src/main/java/com/example/ems/controller/DepartmentController.java`
- `src/main/java/com/example/ems/controller/EmployeeController.java`

Source Code

****File:** `src/main/java/com/example/ems/controller/DepartmentController.java`******

```
package com.example.ems.controller;

import com.example.ems.entity.Department;
import com.example.ems.repository.DepartmentRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/departments")
@RequiredArgsConstructor
public class DepartmentController {
    private final DepartmentRepository repository;

    @PostMapping
    public Department create(@RequestBody Department dept) {
```

```
        return repository.save(dept);
    }

    @GetMapping
    public List<Department> getAll() {
        return repository.findAll();
    }
}
```

****File: src/main/java/com/example/ems/controller/EmployeeController.java****

```
package com.example.ems.controller;

import com.example.ems.entity.Employee;
import com.example.ems.repository.EmployeeRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/employees")
@RequiredArgsConstructor
public class EmployeeController {
    private final EmployeeRepository repository;

    @PostMapping
    public Employee create(@RequestBody Employee emp) {
        return repository.save(emp);
    }

    @GetMapping
    public List<Employee> getAll() {
        return repository.findAll();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        repository.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 4: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
INFO
org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerMapping
: Mapped "{[/api/employees],methods=[POST]}"
INFO
org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerMapping
: Mapped "{[/api/employees],methods=[GET]}"
INFO
org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerMapping
: Mapped "{[/api/employees/{id}],methods=[DELETE]}"
```

Output / Result

Standard RESTful endpoints were successfully bound to HTTP request vectors. End-to-end persistence lifecycles execute predictably across the web tier.

Exercise 5: Employee Management System - Defining Query Methods

Category

Spring Data JPA & JPQL

Aim

To implement advanced search operations using syntactic query derivations, custom JPQL, and named queries.

Concept

The framework interprets method prefixes (**findBy**, **readBy**) to derive SQL predicates. For complex lookups, **@Query** executes explicit JPQL or SQL statements directly.

Scenario

Administrators require specialized retrieval mechanisms to locate workforce members based on domain criteria such as keyword matches or corporate department identities.

Steps Performed

1. Added syntactic query derivation methods to search by containing keywords.
2. Declared an explicit JPQL query utilizing the **@Query** annotation.
3. Configured parameter bindings utilizing the **@Param** annotation.
4. Declared static **@NamedQuery** definitions on domain entities.
5. Exposed query capabilities via dedicated REST service endpoints.

Project / Files Created

- `src/main/java/com/example/ems/repository/EmployeeRepository.java` (Updated)
- `src/main/java/com/example/ems/controller/EmployeeSearchController.java`

Source Code

****File:** `src/main/java/com/example/ems/repository/EmployeeRepository.java` ******

```
package com.example.ems.repository;

import com.example.ems.entity.Employee;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    List<Employee> findByNameContainingIgnoreCase(String keyword);

    @Query("SELECT e FROM Employee e WHERE e.department.name = :deptName")
    List<Employee> findByDepartmentName(@Param("deptName") String deptName);
}
```

****File:** `src/main/java/com/example/ems/controller/EmployeeSearchController.java` ******

```
package com.example.ems.controller;

import com.example.ems.entity.Employee;
import com.example.ems.repository.EmployeeRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/search/employees")
@RequiredArgsConstructor
public class EmployeeSearchController {
    private final EmployeeRepository repository;

    @GetMapping("/name")
    public List<Employee> searchByName(@RequestParam String keyword) {
        return repository.findByNameContainingIgnoreCase(keyword);
    }

    @GetMapping("/department")
    public List<Employee> searchByDept(@RequestParam String dept) {
        return repository.findByDepartmentName(dept);
    }
}
```

```
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 5: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
DEBUG org.hibernate.SQL : select employee0_.id as id1_1_, employee0_.department_id
as departme4_1_, employee0_.email as email2_1_, employee0_.name as name3_1_ from
employees employee0_ where upper(employee0_.name) like upper(?) escape ?
DEBUG org.hibernate.SQL : select employee0_.id as id1_1_, employee0_.department_id
as departme4_1_ from employees employee0_ inner join departments department1_ on
employee0_.department_id=department1_.id where department1_.name=?
```

Output / Result

Advanced query resolution mechanisms operated successfully. Syntactic derivations and JPQL expressions translated accurately into SQL queries with correct runtime parameter bindings.

Exercise 6: Employee Management System - Implementing Pagination and Sorting

Category

Database & Performance Optimization

Aim

To optimize data processing overhead by implementing windowed result pagination and dynamic result sorting.

Concept

Pageable interfaces encapsulate windowing parameters (page size, offset, sorting vectors). Passing **Pageable** objects into repositories forces underlying databases to execute native slice lookups (**LIMIT/OFFSET**).

Scenario

Large enterprise workforces generate massive datasets. Unbounded record retrieval degrades network bandwidth and memory capacity, requiring controlled chunking.

Steps Performed

1. Configured repository endpoints to accept `Pageable` arguments.
2. Created controller vectors accepting standard query parameters (`page`, `size`, `sort`).
3. Instantiated `PageRequest` definitions containing dynamic `Sort` vectors.
4. Invoked repository slicing accessors returning `Page<T>` wrappers.
5. Verified slice boundaries and metadata properties inside response bodies.

Project / Files Created

- `src/main/java/com/example/ems/controller/EmployeePageController.java`

Source Code

****File:** `src/main/java/com/example/ems/controller/EmployeePageController.java` ******

```
package com.example.ems.controller;

import com.example.ems.entity.Employee;
import com.example.ems.repository.EmployeeRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.*;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/paged/employees")
@RequiredArgsConstructor
public class EmployeePageController {
    private final EmployeeRepository repository;

    @GetMapping
    public Page<Employee> getPaged(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "5") int size,
        @RequestParam(defaultValue = "name") String sortBy) {

        Pageable pageable = PageRequest.of(page, size,
Sort.by(sortBy).ascending());
        return repository.findAll(pageable);
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 6: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
DEBUG org.hibernate.SQL : select employee0_.id as id1_1_, employee0_.department_id
as departme4_1_, employee0_.email as email2_1_, employee0_.name as name3_1_ from
employees employee0_ order by employee0_.name asc limit ? offset ?
DEBUG org.hibernate.SQL : select count(employee0_.id) as col_0_0_ from employees
employee0_
```

Output / Result

Result pagination and sorting operated efficiently. Database queries appended structural bounds automatically, optimizing memory consumption and query latency across large result sets.

Exercise 7: Employee Management System - Enabling Entity Auditing

Category

Spring Data JPA & AOP

Aim

To automate lifecycle tracking of persistence entities by capturing historical audit metadata timestamps.

Concept

Entity auditing leverages event listeners to intercept persistence lifecycle events ([PrePersist](#), [PreUpdate](#)). Audit annotations dynamically inject operational timestamps into entities.

Scenario

Regulatory compliance mandates absolute traceability regarding data modifications, requiring automatic capture of creation and alteration timestamps.

Steps Performed

1. Activated framework auditing via [@EnableJpaAuditing](#) configuration.
2. Created an abstract mapped superclass containing audit fields.
3. Annotated fields with [@CreatedDate](#) and [@LastModifiedDate](#).
4. Bound the [AuditingEntityListener](#) to target domain models.
5. Refactored domain models to inherit persistent audit fields.

Project / Files Created

- [src/main/java/com/example/ems/config/AuditConfig.java](#)
- [src/main/java/com/example/ems/entity/Auditable.java](#)

Source Code

****File:** [src/main/java/com/example/ems/config/AuditConfig.java](#)******

```
package com.example.ems.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;

@Configuration
@EnableJpaAuditing
public class AuditConfig {
}
```

****File:** src/main/java/com/example/ems/entity/Auditable.java**

```
package com.example.ems.entity;

import lombok.*;
import org.springframework.data.annotation.*;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;
import javax.persistence.*;
import java.time.LocalDateTime;

@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
@Getter @Setter
public abstract class Auditable {

    @CreatedDate
    @Column(nullable = false, updatable = false)
    private LocalDateTime createdDate;

    @LastModifiedDate
    @Column(nullable = false)
    private LocalDateTime lastModifiedDate;
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 7: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
DEBUG org.hibernate.SQL : insert into employees (created_date, last_modified_date,
department_id, email, name, id) values (?, ?, ?, ?, ?, null)
BIND org.hibernate.type.descriptor.sql.BasicBinder : binding parameter [1] as
[TIMESTAMP] - [2026-06-27T12:20:52]
```

```
BIND org.hibernate.type.descriptor.sql.BasicBinder : binding parameter [2] as
[TIMESTAMP] - [2026-06-27T12:20:52]
```

Output / Result

Persistence auditing operated successfully. Interception listeners captured physical database transactions, injecting accurate temporal metadata without requiring manual assignments inside business services.

Exercise 8: Employee Management System - Creating Projections

Category

Database & Performance Optimization

Aim

To construct structural projections optimizing query execution by retrieving specific attribute subsets.

Concept

Projections restrict database queries to fetch only required columns instead of entire entity tables. Interface-based projections dynamically map query outputs via runtime proxy bindings.

Scenario

High-frequency client dashboard lookups require only employee names and emails. Fetching complete entity trees consumes unnecessary processing overhead.

Steps Performed

1. Created the `EmployeeSummary` interface defining getter contracts.
2. Created a Data Transfer Object class for class-based projection mappings.
3. Declared repository accessors returning collection subsets.
4. Implemented constructor expressions inside custom JPQL queries.
5. Exposed optimized lookup vectors via web controller bindings.

Project / Files Created

- `src/main/java/com/example/ems/projection/EmployeeSummary.java`
- `src/main/java/com/example/ems/projection/EmployeeDto.java`
- `src/main/java/com/example/ems/controller/ProjectionController.java`

Source Code

****File:** `src/main/java/com/example/ems/projection/EmployeeSummary.java` ******

```
package com.example.ems.projection;
```

```
public interface EmployeeSummary {  
    String getName();  
    String getEmail();  
}
```

****File: src/main/java/com/example/ems/projection/EmployeeDto.java****

```
package com.example.ems.projection;  
  
import lombok.*;  
  
@Getter @Setter @AllArgsConstructor  
public class EmployeeDto {  
    private String name;  
    private String departmentName;  
}
```

****File: src/main/java/com/example/ems/controller/ProjectionController.java****

```
package com.example.ems.controller;  
  
import com.example.ems.projection.*;  
import com.example.ems.repository.EmployeeRepository;  
import lombok.RequiredArgsConstructor;  
import org.springframework.web.bind.annotation.*;  
import java.util.List;  
  
@RestController  
@RequestMapping("/api/projections/employees")  
@RequiredArgsConstructor  
public class ProjectionController {  
    private final EmployeeRepository repository;  
  
    @GetMapping("/summary")  
    public List<EmployeeSummary> getSummary() {  
        return repository.findAllProjectedBy();  
    }  
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 8: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
DEBUG org.hibernate.SQL : select employee0_.name as col_0_0_, employee0_.email as col_1_0_ from employees employee0_
```

Output / Result

Projection abstractions executed successfully. The database query retrieved restricted column subsets (`name`, `email`), avoiding unnecessary object hydration across the network boundary.

Exercise 9: Employee Management System - Customizing Data Source Configuration

Category

Spring Boot & Configuration

Aim

To override framework auto-configuration by explicitly establishing data source beans and connection pooling properties.

Concept

While Spring Boot auto-configures default datastores, enterprise scenarios often require custom connection factories. Externalized properties map directly to physical data source implementations like HikariCP.

Scenario

Production deployments require tuning connection pooling behavior (maximum pool size, connection timeouts) to maintain stability under concurrent load spikes.

Steps Performed

1. Externalized connection pool metrics inside application configuration files.
2. Created a dedicated configuration class annotated with `@Configuration`.
3. Instantiated an explicit `DataSource` bean utilizing `@ConfigurationProperties`.
4. Bound connection factory metrics directly to HikariCP pooling structures.
5. Verified connection pool bootstrapping inside runtime application logs.

Project / Files Created

- `src/main/resources/application.properties` (Updated)
- `src/main/java/com/example/ems/config/DataSourceConfig.java`

Source Code

****File: src/main/resources/application.properties****

```
app.datasource.url=jdbc:h2:mem:customdb
app.datasource.driverClassName=org.h2.Driver
app.datasource.username=sa
app.datasource.password=securepass
app.datasource.maximum-pool-size=10
app.datasource.connection-timeout=30000
```

****File: src/main/java/com/example/ems/config/DataSourceConfig.java****

```
package com.example.ems.config;

import com.zaxxer.hikari.HikariDataSource;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.jdbc.DataSourceBuilder;
import org.springframework.context.annotation.*;
import javax.sql.DataSource;

@Configuration
public class DataSourceConfig {

    @Bean
    @Primary
    @ConfigurationProperties(prefix = "app.datasource")
    public DataSource primaryDataSource() {
        return DataSourceBuilder.create()
            .type(HikariDataSource.class)
            .build();
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 9: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
INFO com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
INFO com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Added connection conn0:
url=jdbc:h2:mem:customdb user=SA
```

```
INFO com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
```

Output / Result

Custom data source bootstrapping operated successfully. The application context bypassed default configurations, initializing a HikariCP connection pool tuned to explicit externalized property limits.

Exercise 10: Employee Management System - Hibernate-Specific Features

Category

Hibernate ORM & Performance Optimization

Aim

To leverage Hibernate-specific annotations and configurations to optimize batch insert processing overhead.

Concept

Standard JPA processes persist operations sequentially. Hibernate optimizations group consecutive insert statements into single physical JDBC batches, reducing round-trip latency.

Scenario

Data migration pipelines regularly ingest thousands of employee records simultaneously. Unbatched persistence operations cause severe connection bottlenecks.

Steps Performed

1. Configured JDBC batch size parameters inside application properties.
2. Enabled explicit JDBC batch ordering to optimize insert grouping.
3. Annotated entity models with `@DynamicInsert` and `@DynamicUpdate`.
4. Constructed transactional service methods executing loop persistence.
5. Verified batch execution metrics via trace SQL logs.

Project / Files Created

- `src/main/resources/application.properties` (Updated)
- `src/main/java/com/example/ems/service/BatchService.java`

Source Code

****File:** `src/main/resources/application.properties`******

```
spring.jpa.properties.hibernate.jdbc.batch_size=25
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
```

```
spring.jpa.properties.hibernate.generate_statistics=true
```

****File: src/main/java/com/example/ems/service/BatchService.java****

```
package com.example.ems.service;

import com.example.ems.entity.*;
import com.example.ems.repository.*;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.*;

@Service
@RequiredArgsConstructor
public class BatchService {
    private final EmployeeRepository employeeRepository;
    private final DepartmentRepository departmentRepository;

    @Transactional
    public void executeBulkInsert(int count) {
        Department dept = departmentRepository.save(new Department(null, "Batch
Dept", null));
        List<Employee> batch = new ArrayList<>();

        for (int i = 1; i <= count; i++) {
            batch.add(new Employee(null, "Emp " + i, "emp" + i + "@corp.com",
dept));
        }
        employeeRepository.saveAll(batch);
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and executed successfully. The console output obtained after execution is shown below.

Figure 10: Compilation and execution output [INSERT SCREENSHOT HERE]

Expected Console Output

```
DEBUG org.hibernate.SQL : insert into employees (department_id, email, name, id)
values (?, ?, ?, ?)
INFO org.hibernate.engine.internal.StatisticalLoggingSessionEventListener :
Session Metrics {
    25642 nanoseconds spent acquiring 1 JDBC connections;
```

```
    0 nanoseconds spent releasing 0 JDBC connections;  
    452100 nanoseconds spent executing 2 JDBC batches;  
}
```

Output / Result

Bulk batch processing operated successfully. The underlying Hibernate session grouped persistent inserts into optimized JDBC batches, dramatically reducing execution execution latency across bulk transactions.